

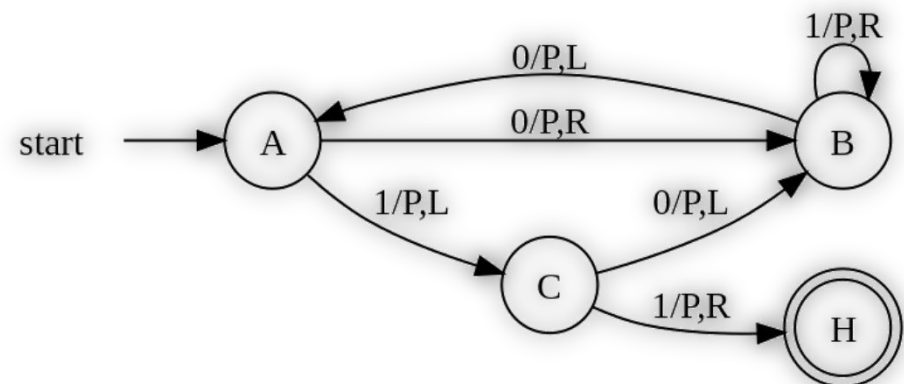
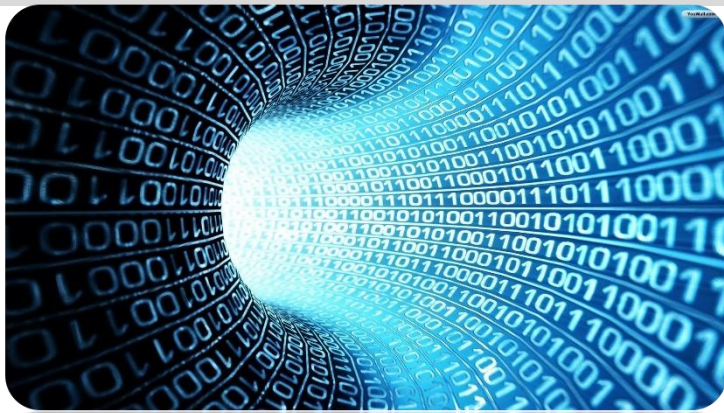
Digital Logic Design and Application

Review

Yan Bo

School of Information & Communication Engineering, UESTC

Email: yanboyu@uestc.edu.cn



Grading

- **Homework:** **5%**
- **Lab** **20%**
- **Final examination:** **75%**

Date: 27 Jun 2026

Time: 9:30-11:30

Content: 25' * 4 Questions

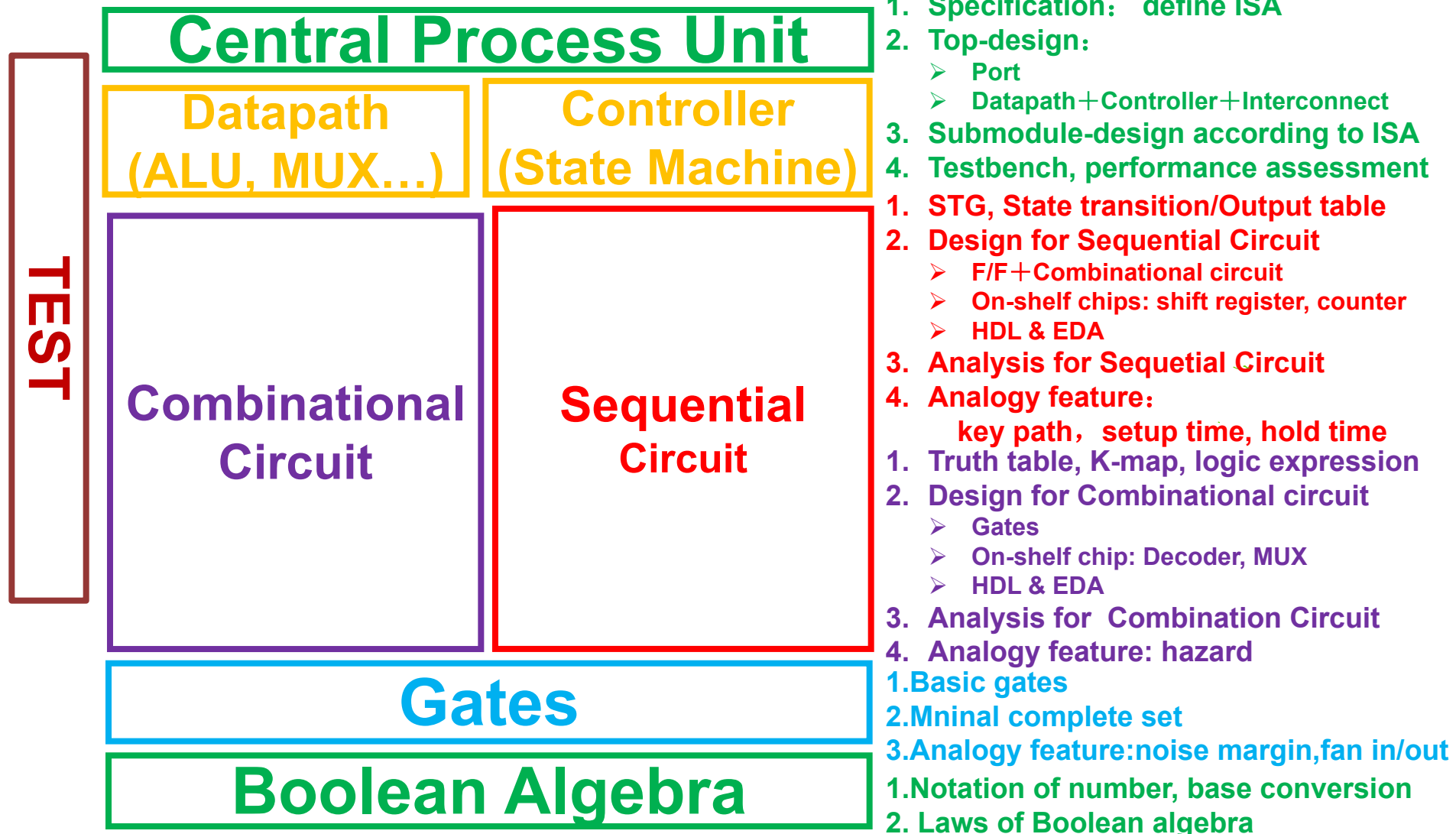
Q1 Multiple choice questions

Q2~Q4 Analysis & Design

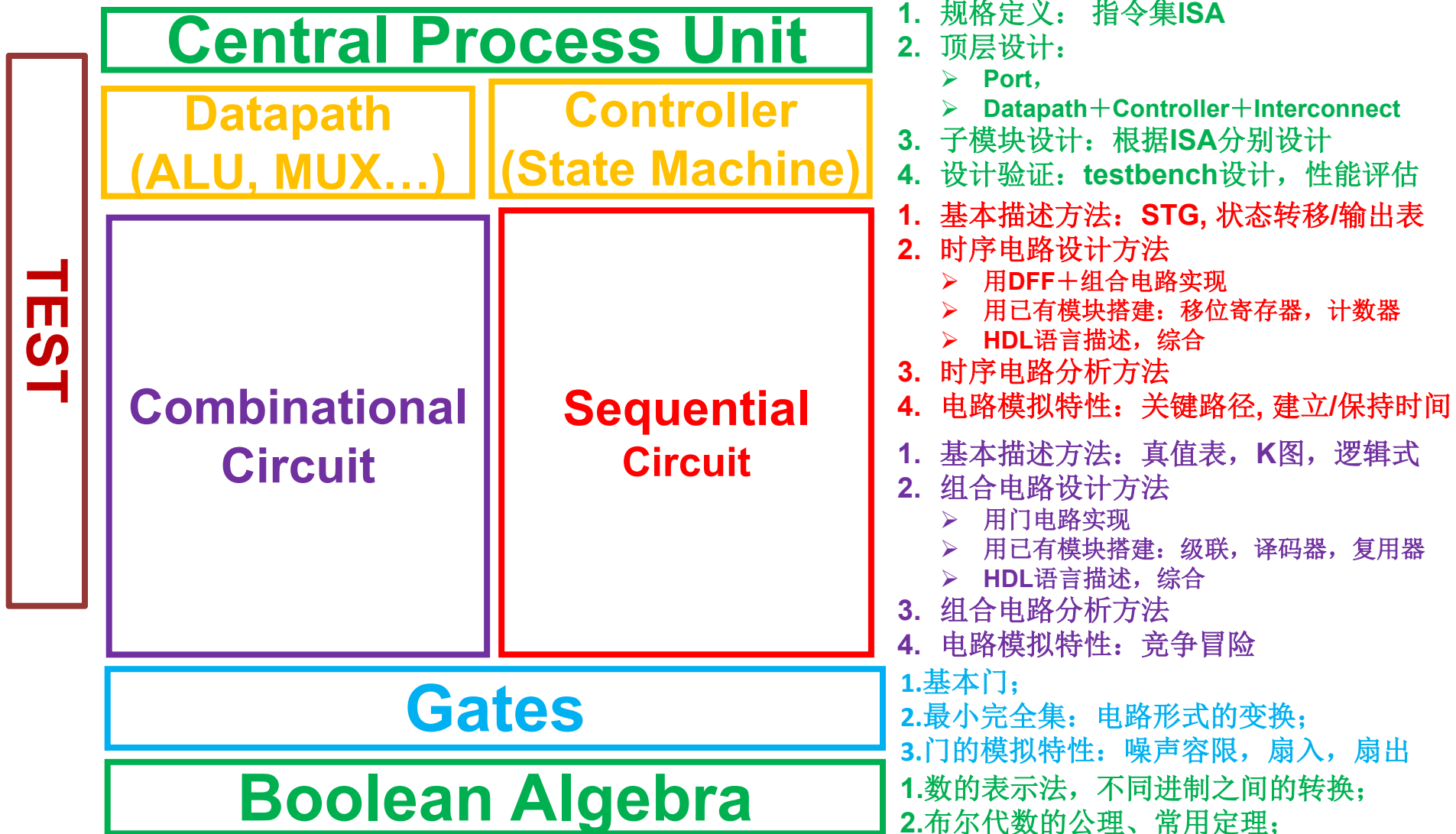
Key Points

- Analogue system & digital system
- Numerical system and Logical Operation
- Digital/logic design, including **combinational circuits**, **sequential circuits**, and timing analysis
- **Verilog** Hardware Description Language
- **Electronic Design Automation** tools
- Von Neumann Computer architecture
- CPU architecture, including datapath and controller
- **CPU design & implement** , including specification, partition& verification

From Boolean Algebra to CPU



From Boolean Algebra to CPU



Digital system & Analogue system: review the base conversion

• Binary to Decimal

Expanded form of polynomial representation

$$(1101.01)_2 = (13.25)_{10}$$

$$= 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 + 0 \times 2^{-1} + 1 \times 2^{-2}$$

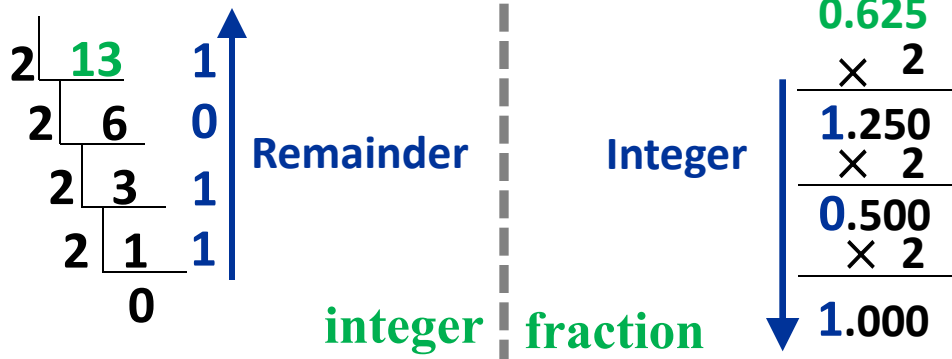
$$= (8)_{10} + (4)_{10} + 0 + (1)_{10} + 0 + (0.25)_{10}$$

• Decimal to Binary

Radix Divide for integer part

Radix Multiply for fraction part

$$(13.625)_{10} = (1101.101)_2$$



Homework chap2

Perform the following expression.

$$1101\ 1110_2 - 0011\ 0010_2$$

$$= (1010\ 1100)_2$$

$$= (AC)_{16} = (254)_8$$

$$= (-84)_{\text{decimal signed number}}$$

Borrow bit= (No)

Overflow bit= (No)

BCD, Gray.....?

BCD code

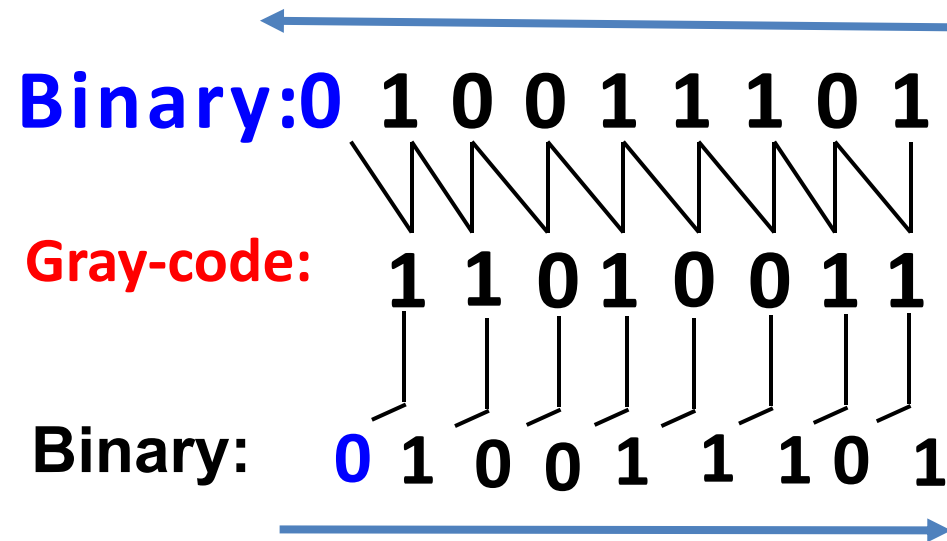
Decimal	BCD Code	2421 Code	Excess-3	Bi-quinary	1-out-of-10
0	0000	0000	0011	0100001	1000000000
1	0001	0001	0100	0100010	0100000000
2	0010	0010	0101	0100100	0010000000
3	0011	0011	0110	0101000	0001000000
4	0100	0100	0111	0110000	0000100000
5	0101	1011	1000	1000001	0000010000
6	0110	1100	1001	1000010	0000001000
7	0111	1101	1010	1000100	0000000100
8	1000	1110	1011	1001000	0000000010
9	1001	1111	1100	1010000	0000000001
unused					
	1010	0101	0000	0000000	0000000000
	1011	0110	0001	0000001	0000000011
	1100	0111	0010	0000010	0000000101
	1101	1000	1101	0000011	0000000110
	1110	1001	1110	0000101	0000000111
	1111	1010	1111	...	...

From Binary code to Gray code

Gray-code	Decimal	Binary
0 0 0	0	000
0 0 1	1	001
0 1 1	2	010
0 1 0	3	011
1 1 0	4	100
1 1 1	5	101
1 0 1	6	110
1 0 0	7	111

Reflecting

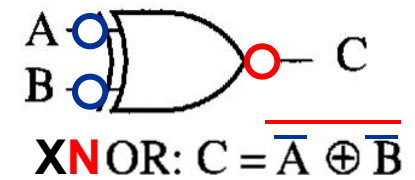
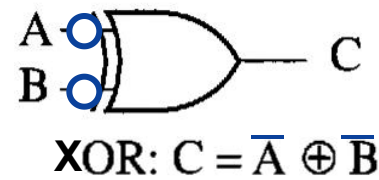
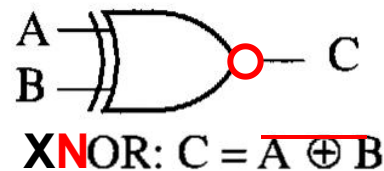
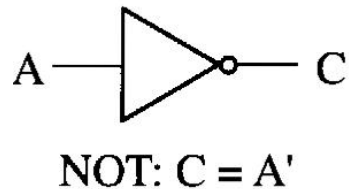
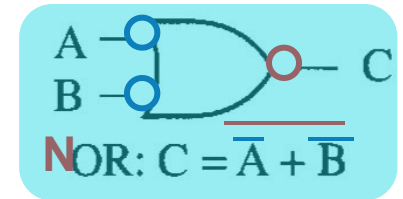
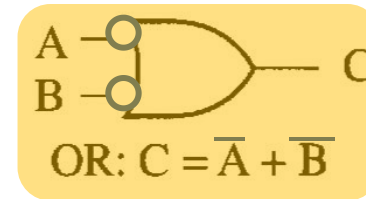
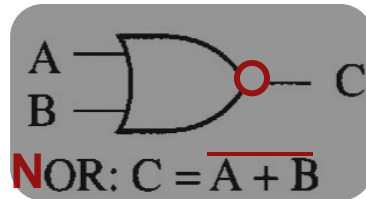
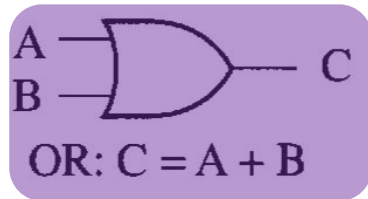
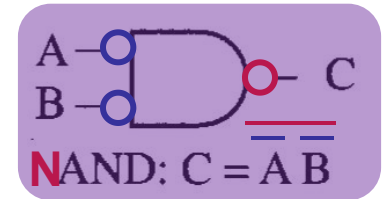
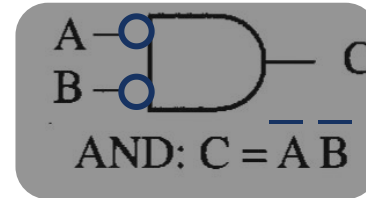
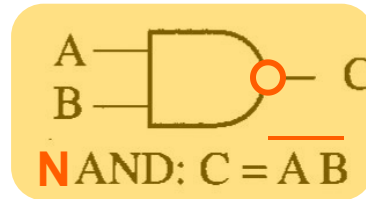
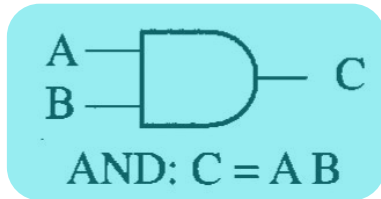
from 1-bit to n-bit



– XOR

from right to left

Gates and Logic: other gates



Minterm List vs. Maxterm List

$$F = \Sigma_{X,Y,Z} (3,5,6)$$

$$F = \Pi_{X,Y,Z} (0,1,2,4,7)$$

$$F' = \Sigma_{X,Y,Z} (0,1,2,4,7)$$

$$F' = \Pi_{X,Y,Z} (3,5,6)$$

$$F^D = \Sigma_{X,Y,Z} (7,6,5,3,0)$$

$$F^D = \Pi_{X,Y,Z} (4,2,1)$$

Homework chap2

Given $F = \Sigma_{ABC} (1,5,7)$, then its complemented expression is $F' = \Sigma_{ABC} (0,2,3,4,6)$, its dual expression is $F^D = \Sigma_{ABC} (1,3,4,5,7)$. And it could be rewrite as $F = \Pi_{ABC} (0,2,3,4,6)$, or $F = A' \Sigma_{BC} (1) + A \Sigma_{BC} (1,3)$.

Gates and Logic: Duality vs. DeMorgan's Law

- **Dual of a logic expression**

Chap2.7.2, 2.7.3

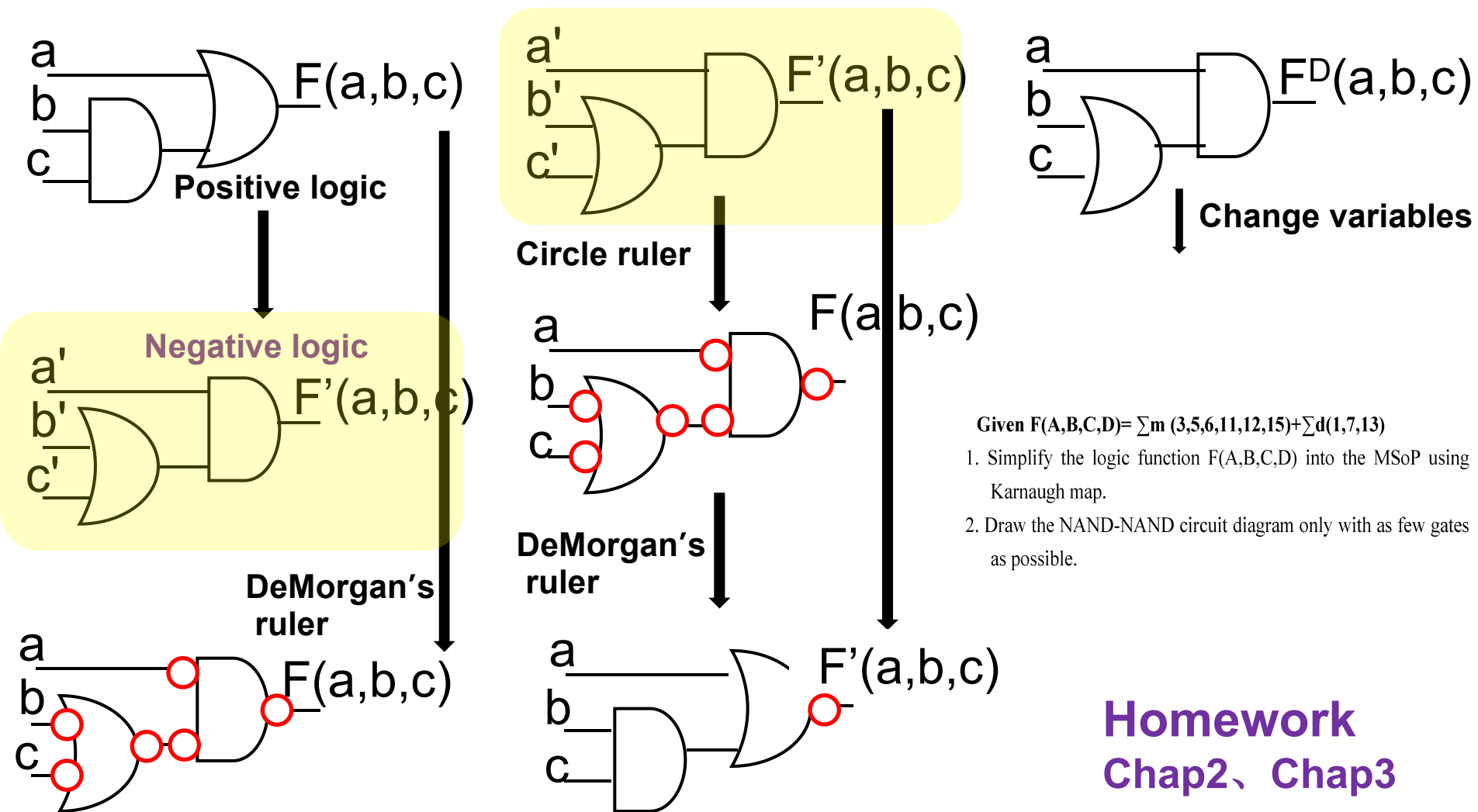
X is not changed !!

$$\begin{aligned} F^D(X_1, X_2, \dots, X_n, 0, 1, +, \cdot) \\ = F(X_1, X_2, \dots, X_n, 1, 0, \cdot, +) \end{aligned}$$

- **Complement of a logic expression**

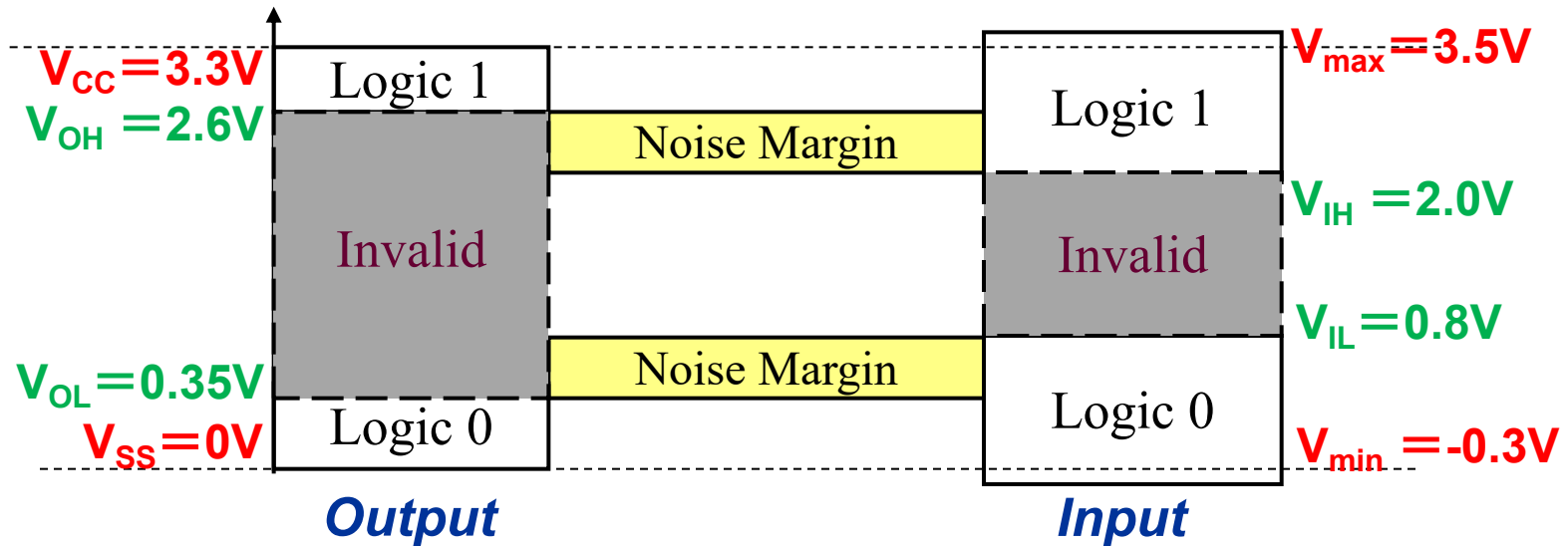
$$\begin{aligned} F'(X_1, X_2, \dots, X_n, 0, 1, +, \cdot) \quad \text{X is inverted !!} \\ = F(X_1', X_2', \dots, X_n', 1, 0, \cdot, +) \end{aligned}$$

Exercise



Digital system & Analogue system:

Digital signals versus Analog signals



A particular inverter has:

$$V_{ILmax} = 0.8V, V_{IHmin} = 2.0V, V_{OLmax} = 0.35V, V_{OHmin} = 2.6V.$$

Then the DC noise margin is (0.6) V in the HIGH state and (0.45)V in the LOW state is.

Explain the meanings of 'Noise margin'. Draw a diagram and show why it is important in the context of the logic design.

Digital system & Analogue system:

Digital signals versus Analog signals

Noise margin : *A measure of how much noise it takes to corrupt a worst-case output voltage into a value that may not be recognized properly by an input.*

Noise margin can ensure that the output signal can be reliably interpret by the following input under different conditions of power-supply voltage, temperature, and output loading

Fan-in : *The number of inputs that a gate can have.*

As the number of input is increased, designers have to increase the size of series transistors to reduces their resistance and the corresponding switching delay. So, typically, the fan-in of CMOS gates is 4 for NOR gates and 6 for NAND gates.

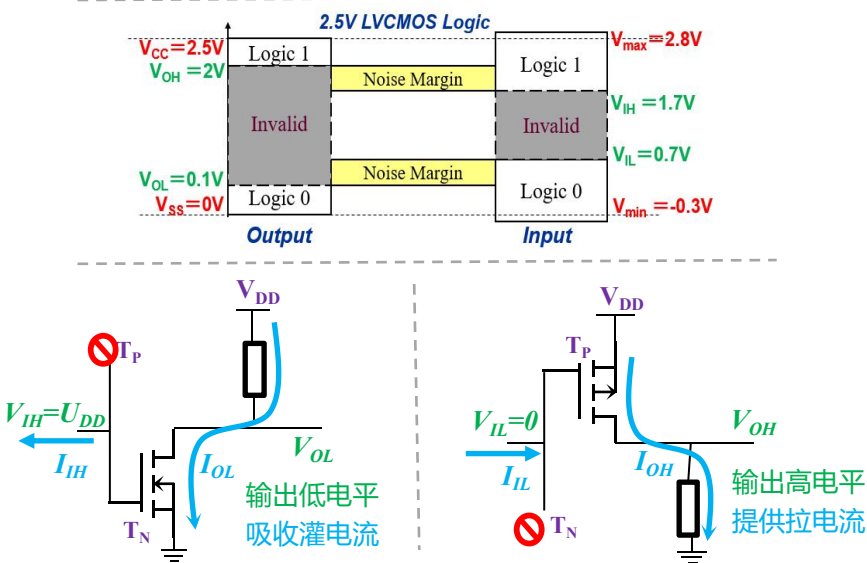
Fan-out : *The number and type of inputs that are connected to a given output.*

Too many inputs connected to an output may affect the speed at which the output changes from one state to another, and also destroy the noise margins.

Electrical Behavior for CMOS device

Static Behavior

- also called "Steady" or "DC" performance
- refers to the performance when the inputs are NOT changing
- **Logic Levels:** V_{OHmin} , V_{OLmax} , V_{IHmin} , V_{ILmax} (Noise Margin)
- **Current:** I_{OLmax} , I_{OHmin} , I_{ILmax} , I_{IHmin} (Fan-In / Fan-Out)



Dynamic Behavior

- also called "Switching" or "AC" performance
- refers to the performance when the inputs are switching
- **Speed:** t_{PD} , t_{PLH} , t_{PHL}
- **Power:** P_{static} , $P_{dynamic}$

Table 3-3 Manufacturer's data sheet for a typical CMOS device, the 54/74HC00 quad NAND gate.

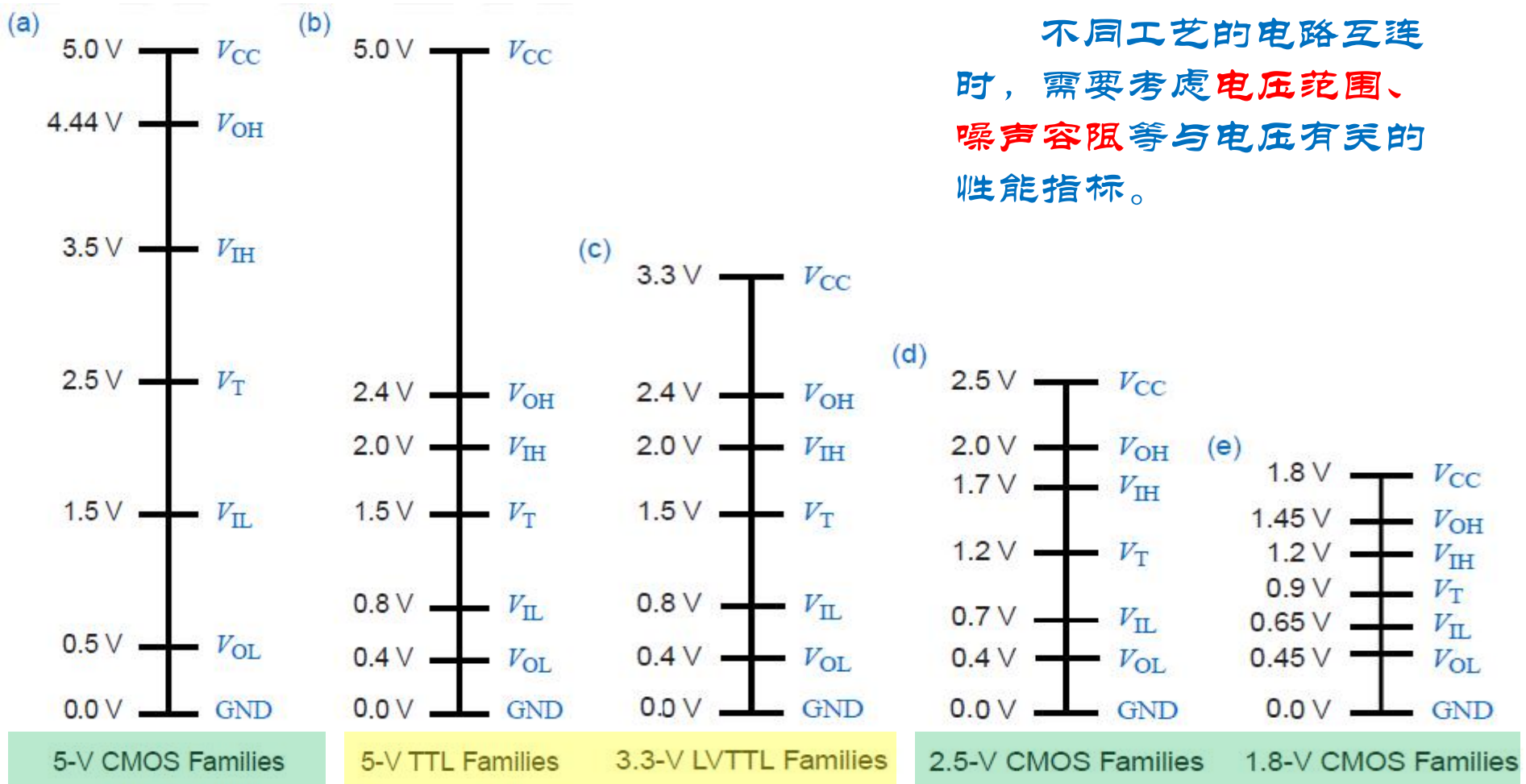
DC ELECTRICAL CHARACTERISTICS OVER OPERATING RANGE						
The following conditions apply unless otherwise specified: Commercial: $T_A = -40^\circ\text{C}$ to $+85^\circ\text{C}$, $V_{CC} = 5.0V \pm 5\%$; Military: $T_A = -55^\circ\text{C}$ to $+125^\circ\text{C}$, $V_{CC} = 5.0V \pm 10\%$						
Sym.	Parameter	Test Conditions ⁽¹⁾	Min.	Typ. ⁽²⁾	Max.	Unit
V_{IH}	Input HIGH level	Guaranteed logic HIGH level	3.15	—	—	V
V_{IL}	Input LOW level	Guaranteed logic LOW level	—	—	1.35	V
I_{IH}	Input HIGH current	$V_{CC} = \text{Max.}$, $V_I = V_{CC}$	—	—	1	μA
I_{IL}	Input LOW current	$V_{CC} = \text{Max.}$, $V_I = 0\text{ V}$	—	—	-1	μA
V_{IK}	Clamp diode voltage	$V_{CC} = \text{Min.}$, $I_N = -18\text{ mA}$	—	-0.7	-1.2	V
I_{OS}	Short-circuit current	$V_{CC} = \text{Max.}$, $I_O = \text{GND}$	—	—	-35	mA
V_{OH}	Output HIGH voltage	$V_{CC} = \text{Min.}$, $V_{IN} = V_{IL}$	4.4	4.499	—	V
V_{OL}	Output LOW voltage	$V_{CC} = \text{Min.}$, $V_{IN} = V_{IH}$	3.84	4.3	—	V
		$I_{OH} = -20\text{ }\mu\text{A}$ $I_{OH} = -4\text{ mA}$ $I_{OL} = 20\text{ }\mu\text{A}$ $I_{OL} = 4\text{ mA}$	—	.001	0.1	V
I_{CC}	Quiescent power supply current	$V_{CC} = \text{Max.}$, $V_{IN} = \text{GND}$ or V_{CC} , $I_O = 0$	—	2	10	μA

SWITCHING CHARACTERISTICS OVER OPERATING RANGE, $C_L = 50\text{ pF}$						
Sym.	Parameter ⁽⁴⁾	Test Conditions	Min.	Typ.	Max.	Unit
t_{PD}	Propagation delay	A or B to Y	—	9	19	ns
C_i	Input capacitance	$V_{IN} = 0\text{ V}$	—	3	10	pF
C_{pd}	Power dissipation capacitance per gate	No load	—	22	—	pF

NOTES:

1. For conditions shown as Max. or Min., use appropriate value specified under Electrical Characteristics.
2. Typical values are at $V_{CC} = 5.0\text{ V}$, $+25^\circ\text{C}$ ambient.
3. Not more than one output should be shorted at a time. Duration of short-circuit test should not exceed one second.
4. This parameter is guaranteed but not tested.

Static Behavior – Voltage



Static Behavior – Non-ideal Input voltage

If V_{IN} not very low, increasing to 1.5V, then:

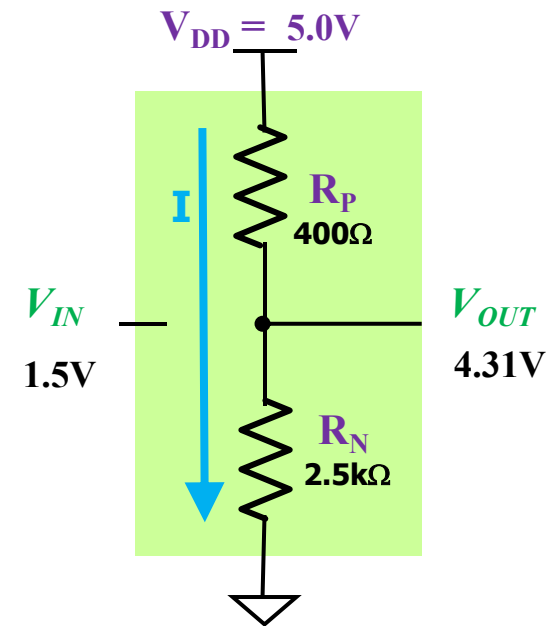
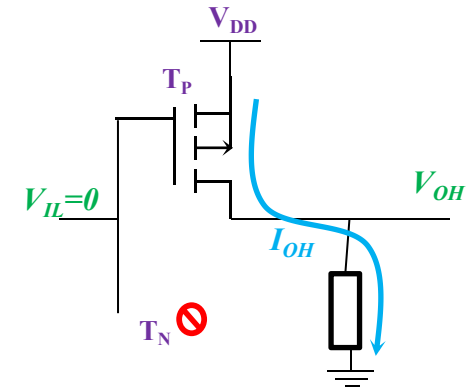
- ✓ R_P is not very small, increasing to 400 Ω
- ✓ R_N is not very large, decreasing to 2.5k Ω

$$I = 5 / (0.4 + 2.5) = 1.72mA$$

$$V_{OUT} = \frac{5}{0.4+2.5} \times 2.5 = 4.31V$$

$$POWER_{Waste} = 5 * I = 8.62mW$$

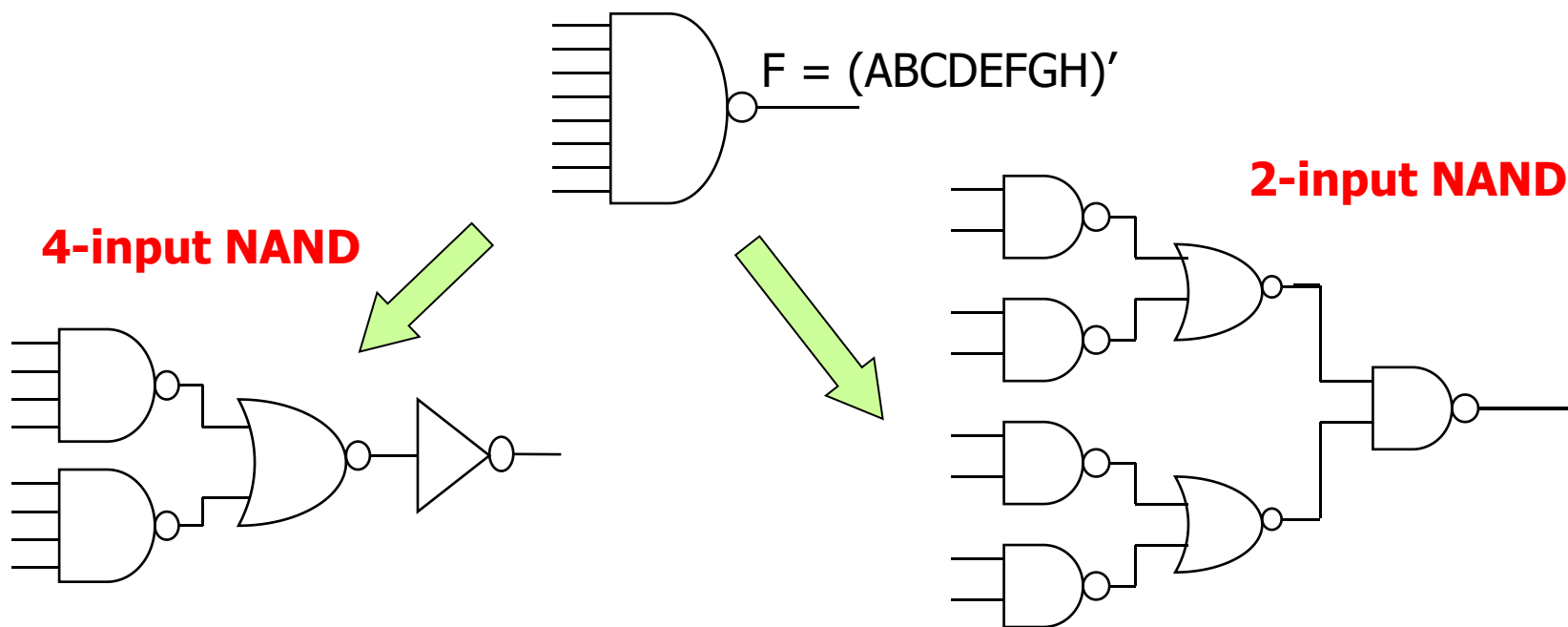
可见，当输入电压不理想时，输出电压也会恶化，并且会消耗更多功耗。



Static Behavior – Fan In

Fan-In: The number of **inputs** that a gate can have.

- Additive “on” Resistance of series transistors limits the Fan-In of gates.
- Typically, Fan-In is **4 for NOR** gates and **6 for NAND** gates.
- A large number of inputs can be fabricated by **cascaded gates** with fewer inputs.



Static Behavior – Fan Out

Fanout: The number of **outputs** that the gate can drive.

扇出系数 $N = \min(\lceil I_{OH}/I_{IH} \rceil, \lceil I_{OL}/I_{IL} \rceil)$

CMOS: 74HCT

$$I_{OH} = -4 \text{ mA}$$

$$I_{OL} = 4 \text{ mA}$$

$$I_{IH} = 1 \mu\text{A}$$

$$I_{IL} = -1 \mu\text{A}$$

TTL: 74LS

$$I_{OH} = -400 \mu\text{A}$$

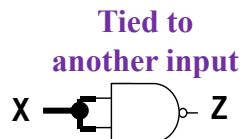
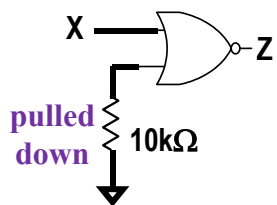
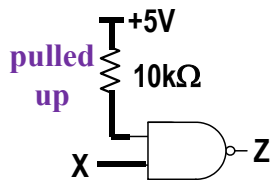
$$I_{OL} = 8 \text{ mA}$$

$$I_{IH} = 20 \mu\text{A}$$

$$I_{IL} = -0.4 \text{ mA}$$

输入高阻，实为漏电流

不用的CMOS输入端绝不能悬空



增加了驱动信号的电容负载，使操作变慢

74HCT Drives 74LS

High Fan-Out $\frac{I_{OH}}{I_{IH}} = \frac{4\text{mA}}{20\mu\text{A}} = 200$

Low Fan-Out $\frac{I_{OL}}{I_{IL}} = \frac{4\text{mA}}{0.4\text{mA}} = 10$

Total Fan-Out $\min(200, 10) = 10$

“excess” driving capability
(高态剩余驱动能力)

$$4\text{mA} - 200\mu\text{A} = 3.8\text{mA}$$

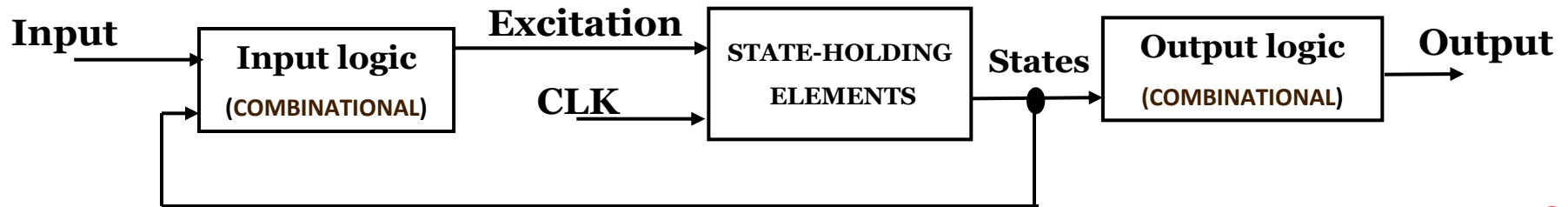
如果负载超出门所能提供的扇出能力:

- 输出低电平时，输出电压 U_{OL} 可能会高于 U_{OLMAX}
- 输出高电平时，输出电压 U_{OH} 可能会低于 U_{OHmin}
- 输出传输延迟增加，电平转换的上升沿和下降沿时间增加。
- 功耗增加，温度上升，可靠性降低，甚至引起器件失效

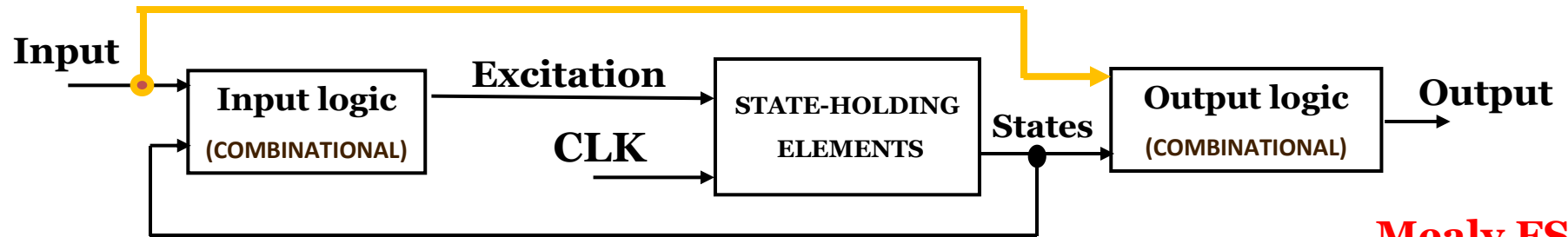
Digital Logic Circuits: Different Types



Combinational Logic



Moore FSM



Mealy FSM

Digital Logic Circuits: Design & Analysis

- **Combinational Circuit**

- **No storage devices**
 - ⇒ The output only depends on the current inputs
- **The most basic component**
 - ⇒ **gates**: NAND, NOR
- **The most commonly used modules**
 - ⇒ **Decoder/Encoder, MUX/DeMUX, ALU**, and so on
 - ⇒ Parity Check circuit, commonly used in communication system

**Homework
Chap4**

- **Sequential Circuit**

- **Combinational circuit + storage devices**
 - ⇒ The output depends on the current as well as past inputs(STATE)
 - ⇒ **Mealy** state machine: the output depends on current input and STATE
 - ⇒ **Moore** state machine: the output only depends on the STATE
- **The most basic component**
 - ⇒ **Flip/Flop**: DFF
- **The most commonly used modules**
 - ⇒ **Register, Counter**, and so on
 - ⇒ **Serial Signal Detector, Serial Signal Generator**

**Homework
Chap5**

Combinational Digital Circuit Design

1.Description: 3-input Majority function, which output is 1 whenever majority of inputs is 1.

2.Truth table

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

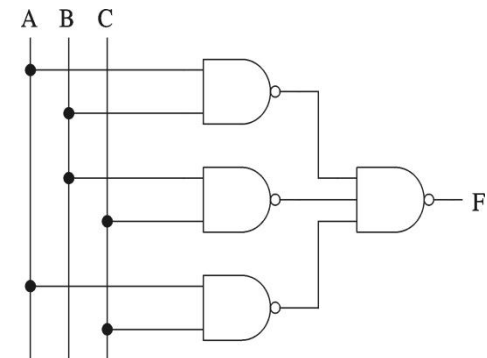
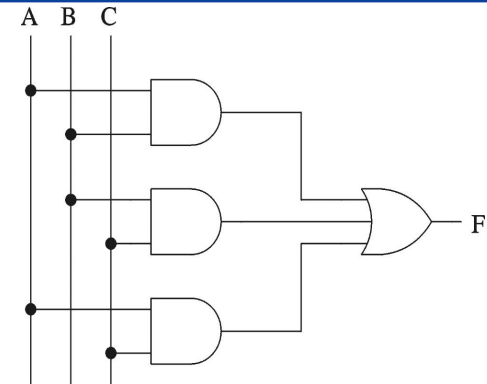
3.K map

AB		C			
		00	01	11	10
C	0	0	0	1	0
	1	0	1	1	1

4.Logical expression

$$F = AB + BC + AC$$

5.Circuit schematic



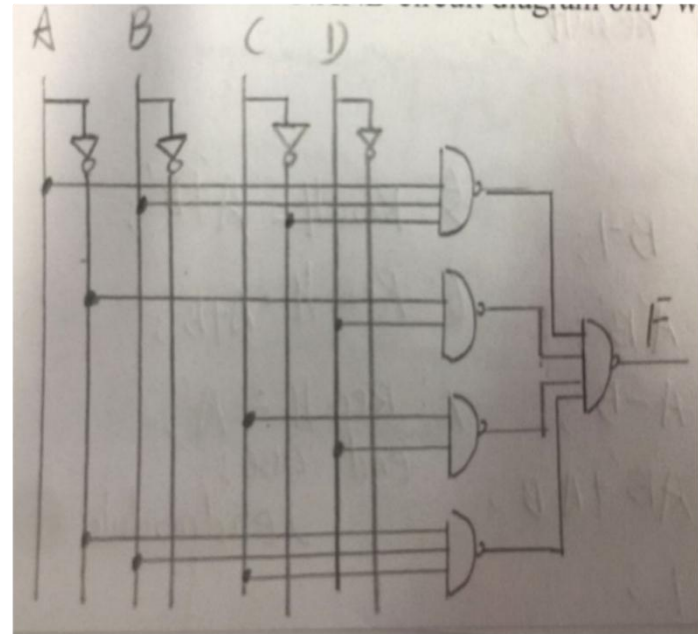
Combinational Digital Circuit

Given $F(A,B,C,D) = \sum m(3,5,6,11,12,15) + \sum d(1,7,13)$

1. Simplify the logic function $F(A,B,C,D)$ into the MSoP using Karnaugh map.
2. Draw the NAND-NAND circuit diagram only with as few gates as possible.

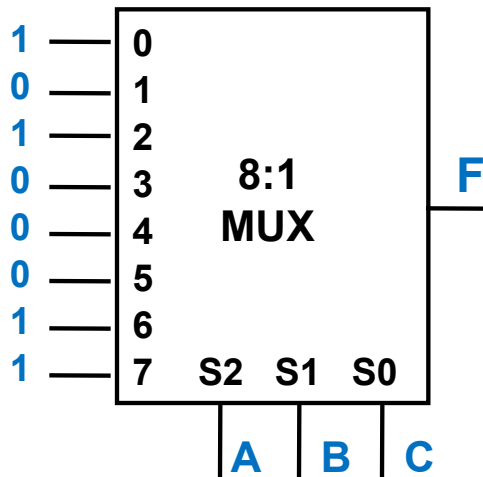
		AB			
		00	01	11	10
CD	00			1	
	01	d	1	d	
	11	1	d	1	1
	10		1		

$$F = CD + BD + A'BC + ABC'$$



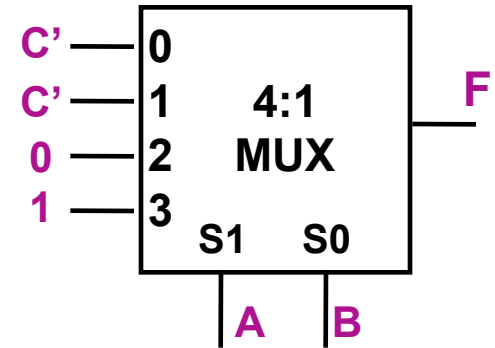
Logic design with MUX

$$F(A,B,C) = m_0 + m_2 + m_6 + m_7$$



Lookup Table

A	B	C	F
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

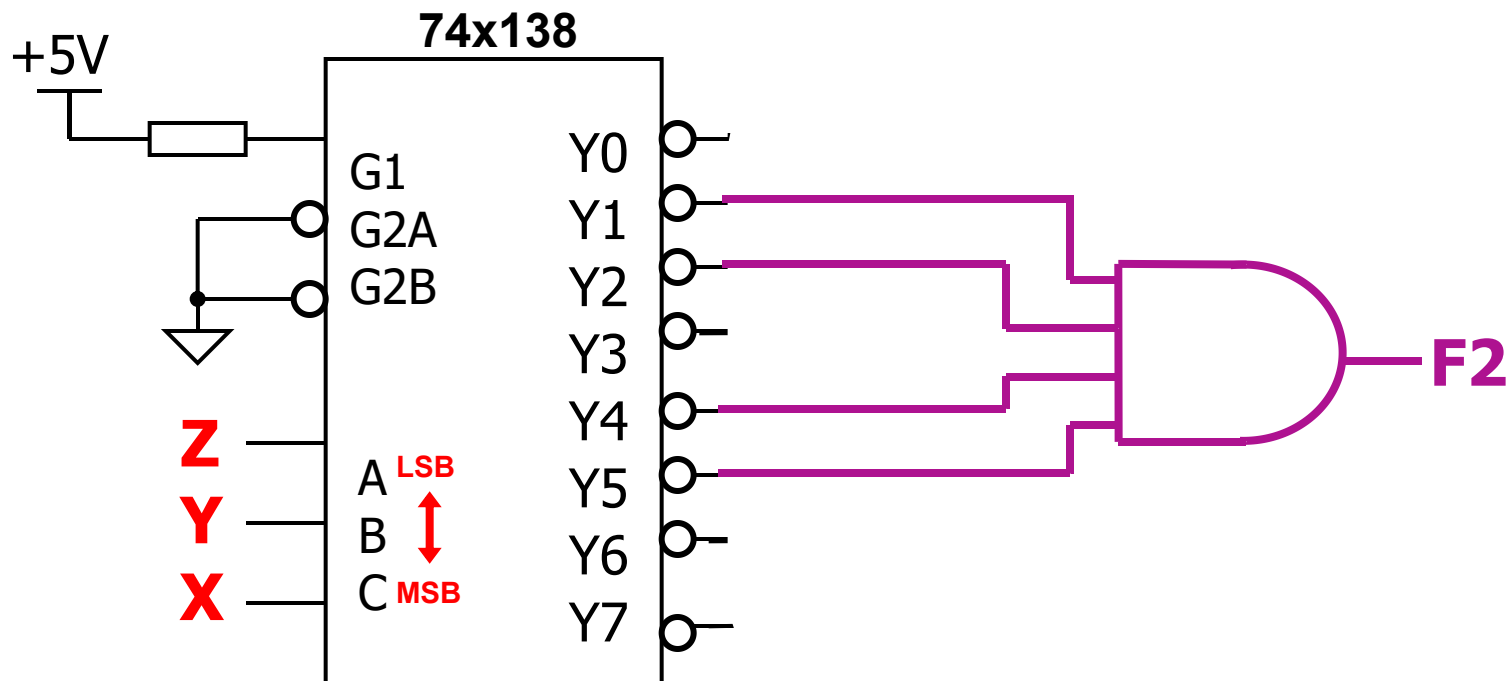


$2^{n-1}:1$ MUX can implement any function of **less than n variables**

Logic design with Decoder and Gates

$$F1 = \sum_{(x,y,z)} (0,3,6,7)$$

While enable input asserted, $Y_i = m_i' = M_i$

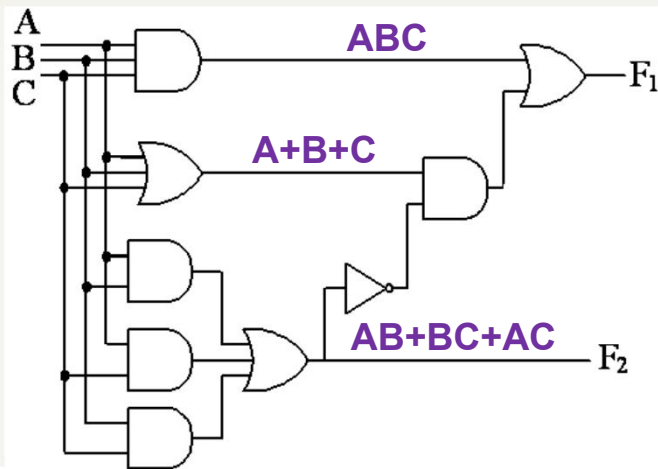


$$F2 = \prod_{(x,y,z)} (1, 2, 4, 5) = M_1 \cdot M_2 \cdot M_4 \cdot M_5 = m_1' \cdot m_2' \cdot m_4' \cdot m_5'$$

Combinational Circuits Analysis

Find a simplified logic for the following circuit :

1. Circuit schematic



2. Logical expression

$$F1 = ABC + (A + B + C)(AB + BC + AC)'$$

$$F2 = AB + BC + AC$$

3. Truth table

A	B	C	F1	F2
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Parity Generator/Checker
(Even Checker)

4. K map

AB \ C	00	01	11	10
0	0	1	0	1
1	1	0	1	0

$F1 = ABC + A'B'C + A'BC' + AB'C'$

AB \ C	00	01	11	10
0	0	0	1	0
1	0	1	1	1

$F2 = AB + BC + AC$

Majority circuit

5. Description: What's the circuit's function?

Basic Verilog Modeling

module is a most basic building block of a design.

port declarations describes the interface of the component, i.e. input and output ports.

circuit functionality describes an implementation of the corresponding module.

```
module module_name (port_name1, port_name2, ...);
```

port declarations

Interconnect declarations

circuit functionality

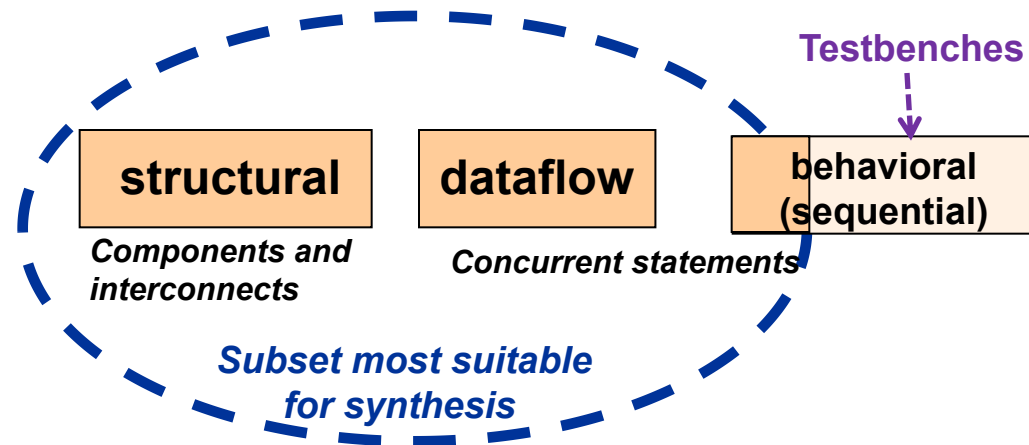
```
endmodule
```

```
port_mode port_name1;  
port_mode port_name2;  
.....
```

```
wire interconnect_name1;  
wire interconnect_name1;  
.....
```

```
assign statements;  
always @(sensitive signals list)  
begin  
    functionality code  
end
```

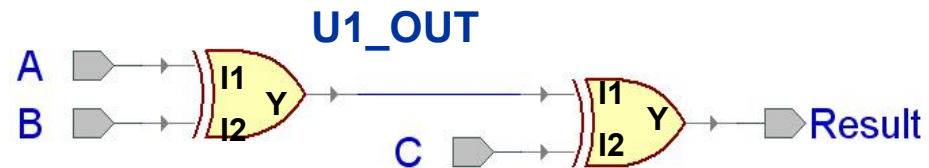
Verilog Design Styles



```
module structural (Result, A, B, C);  
    output Result;  
    input A, B, C;  
    wire U1_OUT;  
  
    xor U1 (U1_OUT, A, B);  
    xor U2 (Result, U1_OUT, C);  
  
endmodule
```

```
module dataflow (Result, A, B, C);  
    output Result;  
    input A, B, C;  
  
    assign Result = (A XOR B) XOR C;  
  
endmodule
```

```
module behavioral (Result, A, B, C);  
    output reg Result;  
    input A, B, C;  
  
    always @ (A, B, C)  
    begin  
        Result = (A XOR B) XOR C;  
    end  
  
endmodule
```



Verilog for Combinational Circuit

```
module Code42( F, I );
```

```
  output [1:0] F;
```

```
  input  [3:0] I;
```

```
  reg    [1:0] t;
```

```
  assign F = t ;
```

```
  always @ ( I )
```

```
    begin
```

```
      case ( I )
```

```
        1:      t = 0 ;
```

```
        2:      t = 1 ;
```

```
        4:      t = 2 ;
```

```
        8 :      t = 3 ;
```

```
      default: t = 0 ;
```

```
    endcase
```

```
  end
```

```
endmodule;
```

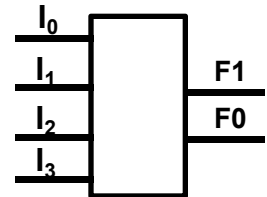
No priority

4-2Priority Encoder

I3	I2	I1	I0	F1	F0
1	X	X	X	1	1
0	1	X	X	1	0
0	0	1	X	0	1
0	0	0	1	0	0

4-2 Encoder

I3	I2	I1	I0	F1	F0
1	0	0	0	1	1
0	1	0	0	1	0
0	0	1	0	0	1
0	0	0	1	0	0



```
  always @ ( I )
```

```
    begin
```

```
      if (I[3]) t= 2'b11 ;
```

```
    else if (I[2]) t = 2'b10 ;
```

```
    else if (I[1]) t = 2'b01 ;
```

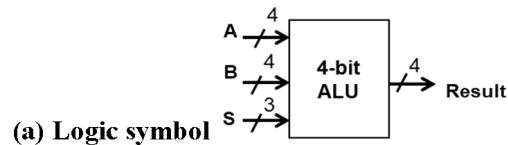
```
    else          t = 2'b00 ;
```

```
  end
```

Priority!

Verilog for Combinational Circuit

Describe a 4-bit ALU in HDL language. The following Fig. is for the logic symbol and the function table.



```

module ALU4 ( output reg [3:0] Result
              input [3:0] A, B,
              input [2:0] S );
    always @( * )
    begin
        case (S)
            0: Result = B - 1;
            1: Result = ~(A | B);
            2: Result = A - B;
            3: Result = ~(A ^ B);
            4: Result = 1;
            5: Result = ~(A & B);
            6: Result = A + B;
            7: Result = ~A;
            default: Result = 8'bX;
        endcase
    end
endmodule

```

(b) Function table

s_2	s_1	s_0	Operations
0	0	0	$B - 1$
0	0	1	$A \text{ NOR } B$
0	1	0	$A - B$
0	1	1	$A \text{ XNOR } B$
1	0	0	1
1	0	1	$A \text{ NAND } B$
1	1	0	$A + B$
1	1	1	A'

Logical

!	Not
&&	and
	Or
~	bit not
&	bit and
	bit or
^	bit xor
<<	shift left
>>	shift right

Arithmetic

+	addition
-	Subtraction
*	Multiply
/	divide
%	modulus

Relational

==	equal
!=	not equal
<	less than
<=	less than or equal
>	greater than
>=	greater than or equal

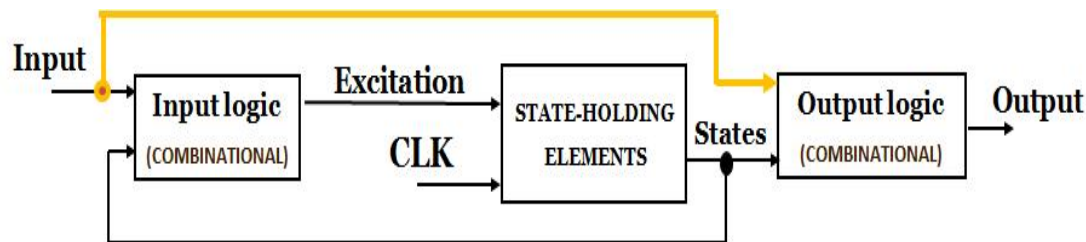
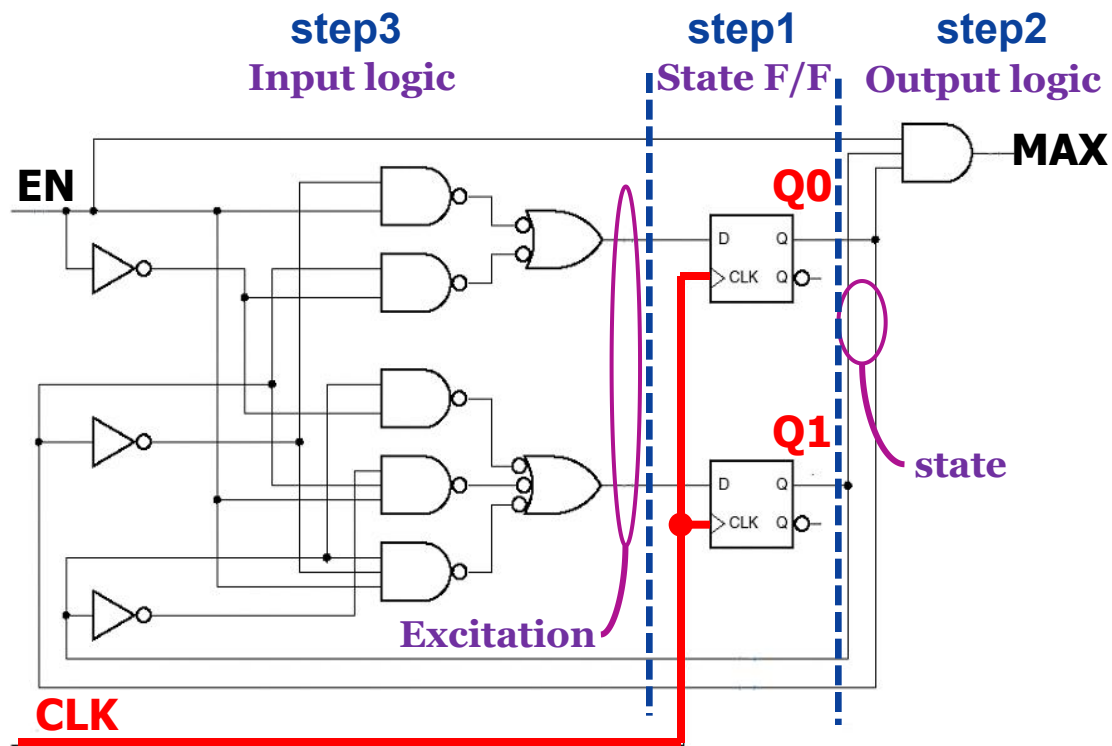
Design & Analysis of Sequential Circuits

Analysis Steps:

1. Input logic equations
2. Output logic equations
3. State transition equations
4. State transition/output table
5. State transition diagram, waveform, etc.

Design Steps:

1. State transition graph
2. State transition/output table
3. State minimization & coding
4. Chose Storage device (DFF)
5. Excitation equations
6. Output equations
7. Logic circuit diagram



FSM Analysis

1. Analyze the FSM shown in Fig.Q3. Write the state transition and output tables and sketch the state transition diagram.

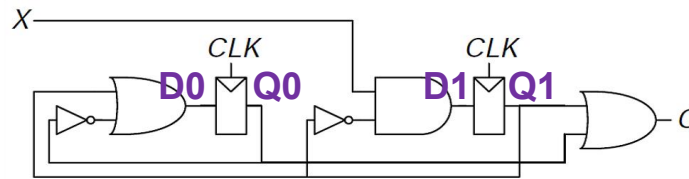


Fig.Q3

2. Is this a Moore or Mealy machine? Describe in words what the FSM does.

1). Excitation equations

$$D0 = Q1 + Q0'$$

$$D1 = X \cdot Q1'$$

2). Output equations

$$Q = Q1 + Q0$$

3). State transition equations

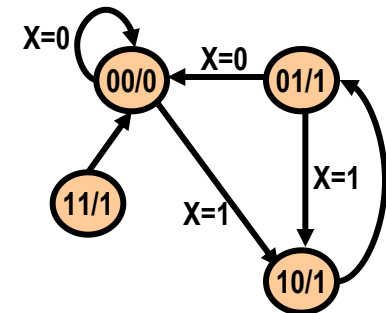
$$Q0+ = D0 = Q1 + Q0'$$

$$Q1+ = D1 = X \cdot Q1'$$

4). State transition /output table

S	X		Q
	0	1	
0 0	00	10	0
0 1	00	10	1
1 0	01	01	1
1 1	00	00	1
Q1Q0	Q1+Q0+		

5). STG



It is a Moore machine:

if $X=1$ then the state transits between 01 and 10, the output Q should be 1;

if $X=0$ then the state reset to 00, the output Q should be 0.

FSM Analysis

1. Excitation equations from input logic

$$D0 = Q0 \cdot EN' + Q0' \cdot EN$$

$$D1 = Q1 \cdot EN' + Q1' \cdot Q0 \cdot EN + Q1 \cdot Q0' \cdot EN$$

2. Output equations

$$MAX = Q1 \cdot Q0 \cdot EN$$

3. State transition equations

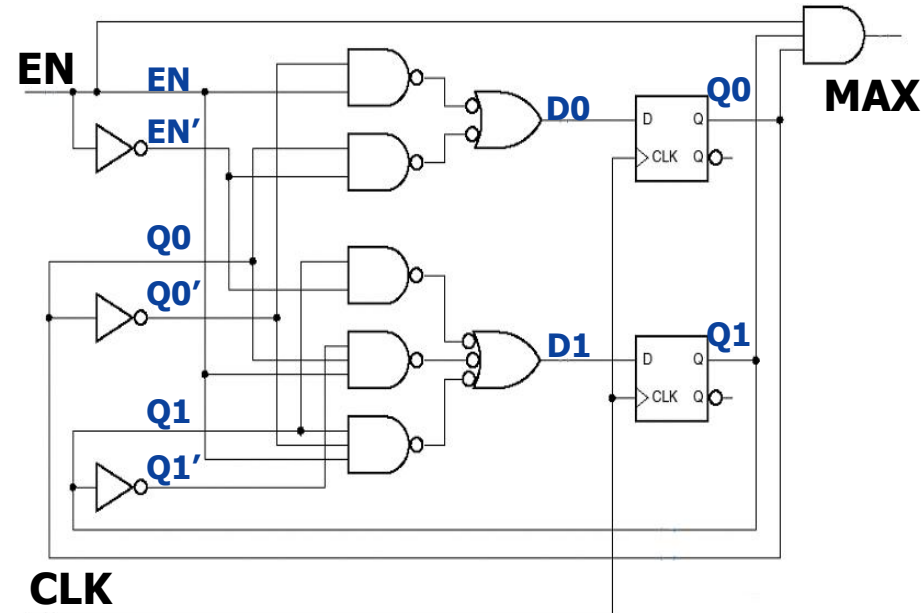
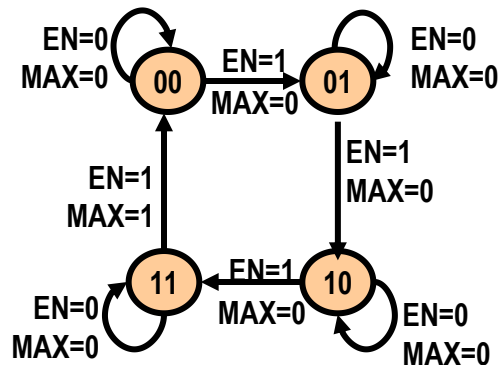
$$Q0+ = D0 = Q0 \cdot EN' + Q0' \cdot EN$$

$$Q1+ = D1 = Q1 \cdot EN' + Q1' \cdot Q0 \cdot EN + Q1 \cdot Q0' \cdot EN$$

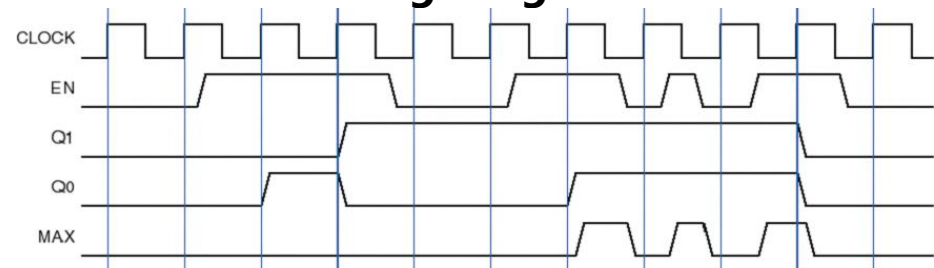
4. State transition /output table

S	EN	
	0	1
0 0	00, 0	01, 0
0 1	01, 0	10, 0
1 0	10, 0	11, 0
1 1	11, 0	00, 1
Q1Q0	Q1+Q0+, MAX	

5. State diagram



6. Timing Diagram



2-bit Add-up Counter with Enable

FSM Design

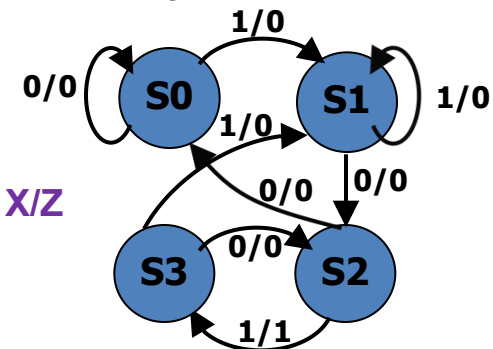
Design a Serial Data Detector of 101 (Mealy machine)

Chap6.4, 6.5

Input $X=010110100110101110.....$

Output $Z=000100100000101000.....$

1. Mealy STG



2. State /output table

S	X	
	0	1
S0	S0, 0	S1, 0
S1	S2, 0	S1, 0
S2	S0, 0	S3, 1
S3	S2, 0	S1, 0
S+, Z		

3. State minimization & coding

S	X	
	0	1
S0	S0, 0	S1, 0
S1	S2, 0	S1, 0
S2	S0, 0	S1, 1
S+, Z		

S	X	
	0	1
S0	00, 0	01, 0
S1	01, 0	01, 0
S2	00, 0	01, 1
Q2Q1 Q2+Q1+, Z		

4. Excitation equations

X	Q2Q1				Q2+
	00	01	11	10	
0	0	1	X	0	
1	0	0	X	0	

X	Q2Q1				Q1+
	00	01	11	10	
0	0	0	X	0	
1	1	1	X	1	

5. Output equations

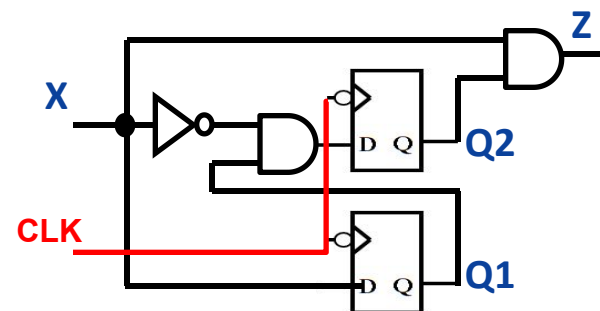
X	Q2Q1				Z
	00	01	11	10	
0	0	0	X	0	
1	0	0	X	1	

$$Q2+=X'Q1$$

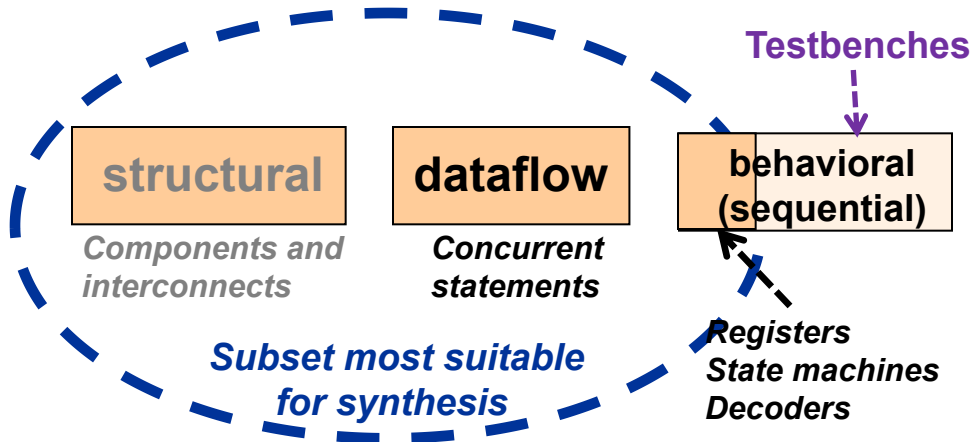
$$Q1+=X$$

$$Z=Q2X$$

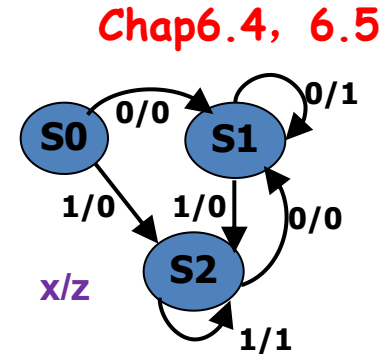
6. Logic circuit diagram (DFF)



Verilog Design Styles: Sequential circuit/FSM



state	X	
	0	1
S0	0 0	01, 0
S1	0 1	01, 1
S2	1 0	01, 0
Y1Y0	Y1+Y0+, z	



```

module behavioral
#( parameter S0=0,S1=1,S2=2 )
(output reg z, input x, clk);
reg [1:0] state, next_state;

always @ ( negedge clk )
begin
    Update State (Sequential circuit)
end

always @ ( state or x )
begin
    Next state & Output (Combinational) circuit
end
endmodule
    
```

state <= next_state;

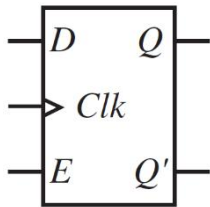
```

case (state)
S0 : if (x==0) begin
        next_state=S1; z=0; --PRESENT STATE/INPUT
    end
    else
        begin
            next_state=S2; z=0; --NEXT STATE/OUTPUT
        end
S1 : if (x==0) begin
        next_state=S1; z=1;
    end
    else
        begin
            next_state=S2; z=0;
        end
S2 : if (x==0) begin
        next_state=S1; z=0;
    end
    else
        begin
            next_state=S2; z=1;
        end
default:
    begin
        next_state=S0; z=0;
    end
end case;
    
```

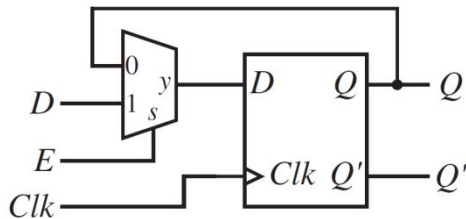
Q1: Is it a Moore or Mealy machine?
Q2: What's the meaning of default branch?

D-type Flip-Flops with other inputs

Logic Symbol



Circuit



Function table

Clk	E	D	Q	Q _{next}
0	×	×	0	0
0	×	×	1	1
1	×	×	0	0
1	×	×	1	1
↑	0	×	0	0
↑	0	×	1	1
↑	1	0	×	0
↑	1	1	×	1

Verilog code

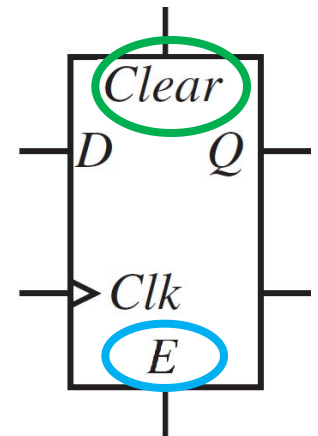
```

module DFF1 (
    input      CLK, D, E, Clear,
    output reg Q      );

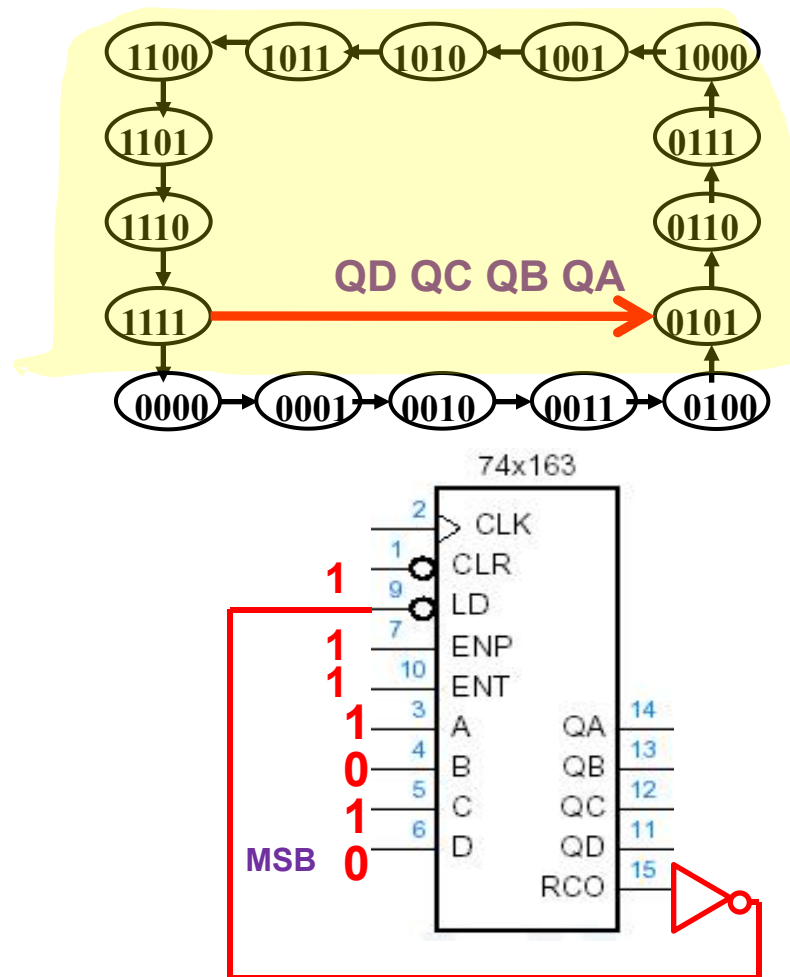
    always@ (posedge CLK or posedge Clear)
    begin
        if ( Clear )
            Q <= 0 ;
        else if ( E )
            Q <= D ;
    end
endmodule
    
```

A D flip-flop with
asynchronous Clear
& synchronous Enable

Logic Symbol

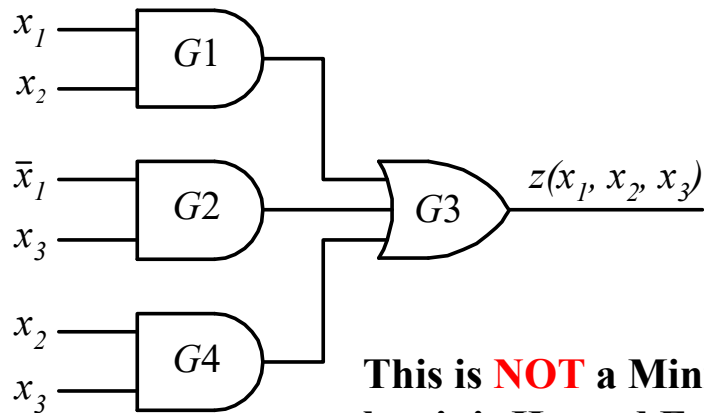
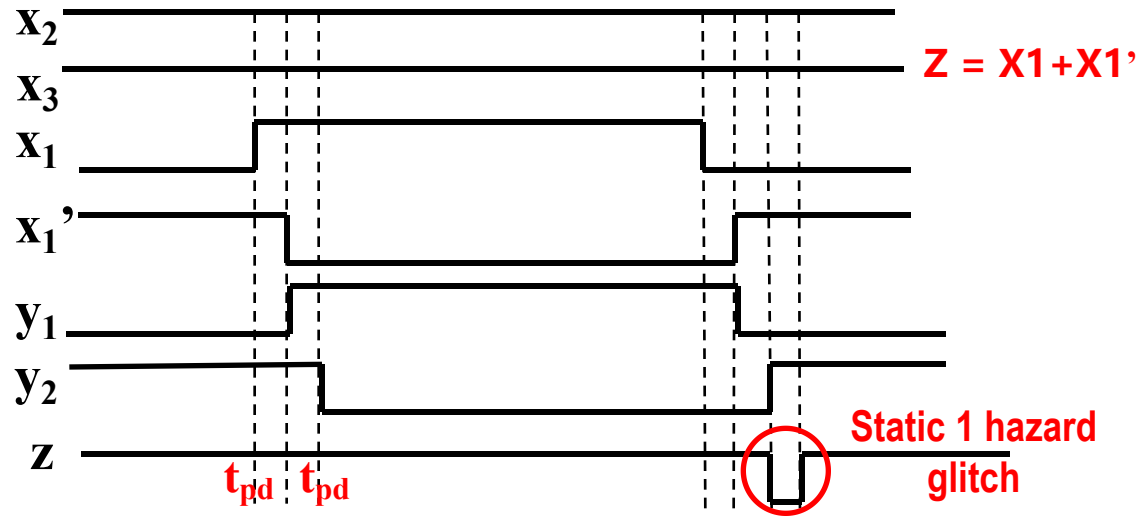
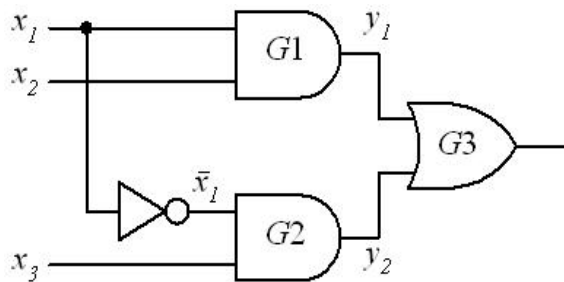


Please design a modulo-11 counter by 74x163.

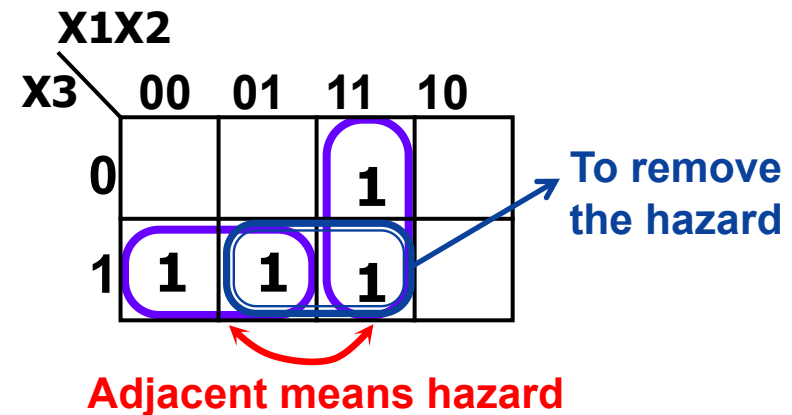


Static hazard in Combinational Circuits

$$Z = x_1 x_2 + x_1' x_3$$



This is **NOT** a Minimal Sum,
but it is Hazard Free.



Timing in Sequential Circuit (DFF)

Propagation delay: $t_{pLH(CQ)}$, $t_{pHL(CQ)}$

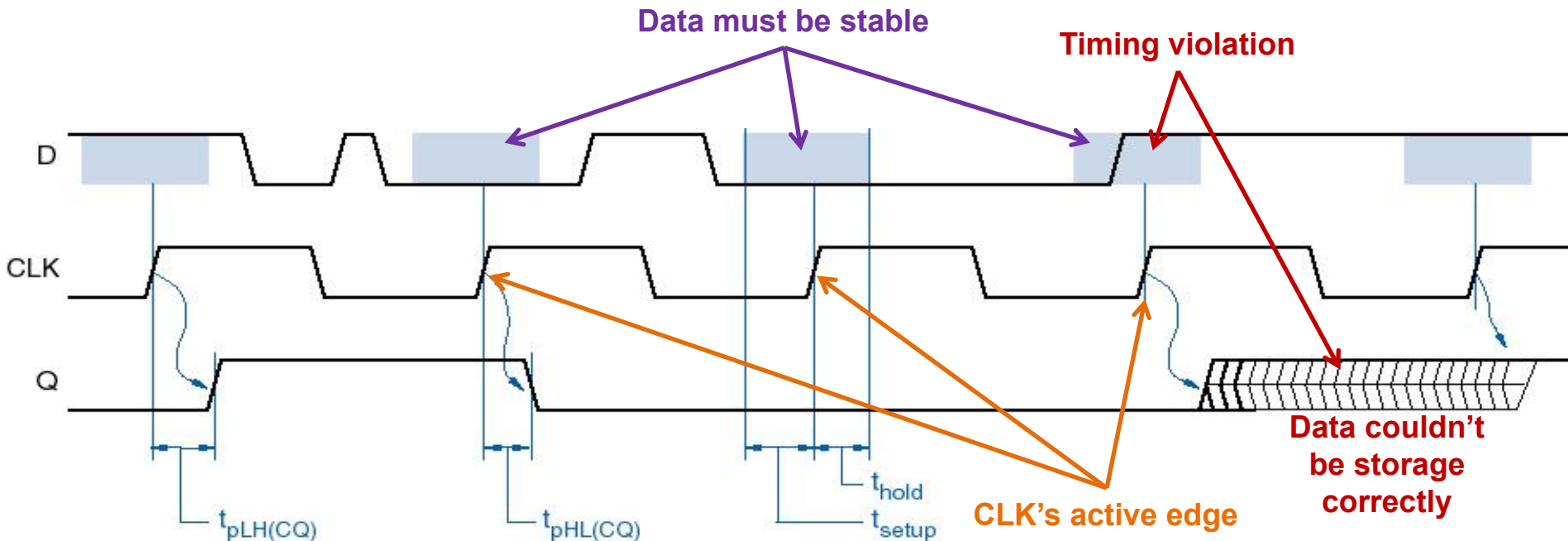
The amount of time from the CLK changed to the output Q changed.

Setup time: t_{setup}

The minimum amount of time BEFORE the CLK's active edge during which the data must be stable.

Hold time: t_{hold}

The minimum amount of time AFTER the clock's active edge by which the data must be stable.



ISA of our CPU

ISA	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Example
	Opcode			*1	*2	Rs1/Rd		Rs2		#imm/offset							
MOV <Rd>, <Op2> ;Rd ← Op2	0	0	0	0	x	Rd		Rs2		x	x	x	x	x	x	x	MOV R0 , R1
	0	0	0	1	x	Rd		x	x	#imm							MOV R0 , #0x22
ADD <Rs1/Rd> ,<Op2> ; Rd ← Rs1+ Op2	0	0	1	0	0	Rs1/Rd		Rs2		x	x	x	x	x	x	x	ADD R0 , R1
	0	0	1	1	0	Rs1/Rd		x	x	#imm							ADD R0 , #0x22
ADDC <Rs1/Rd> ,<Op2> ; Rd ← Rs1+ Op2+C	0	0	1	0	1	Rs1/Rd		Rs2		x	x	x	x	x	x	x	ADDC R0 , R1
	0	0	1	1	1	Rs1/Rd		x	x	#imm							ADDC R0 , #0x22
SUB <Rs1/Rd> ,<Op2> ; Rd ← Rs1- Op2	0	1	0	0	0	Rs1/Rd		Rs2		x	x	x	x	x	x	x	SUB R0 , R1
	0	1	0	1	0	Rs1/Rd		x	x	#imm							SUB R0 , #0x22
SUBB <Rs1/Rd> ,<Op2> ; Rd ← Rs1-Op2-C	0	1	0	0	1	Rs1/Rd		Rs2		x	x	x	x	x	x	x	SUBB R0 , R1
	0	1	0	1	1	Rs1/Rd		x	x	#imm							SUBB R0 , #0x22
AND <Rs1/Rd> ,<Rs2> ; Rd ← Rs1· Rs2	0	1	1	0	x	Rs1/Rd		Rs2		x	x	x	x	x	x	x	AND R0 , R1
NOT <Rs1/Rd> ;Rs1← !Rs1	1	0	0	0	x	Rs1/Rd		x	x	x	x	x	x	x	x	x	NOT R0
JMP <Rs> ;PC ← Rs	1	0	1	x	0	x	x	Rs		x	x	x	x	x	x	x	JMP R0
JZ <offset> ;PC ← PC+offset (ZF=1)	1	0	1	0	1	x	x	x	x	offset							JZ XXX
JC <offset> ;PC ← PC+offset(CF=1)	1	0	1	1	1	x	x	x	x	offset							JC XXX
LDR <Rd>, <Rs> ;Rd ← [Rs]	1	1	0	x	x	Rd		Rs		x	x	x	x	x	x	x	LAD R0, R1
STR <Rd>, <Rs> ;[Rd] ← Rs	1	1	1	x	X	Rd		Rs		x	x	x	x	x	x	x	STR R0, R1

*1: <op2> is in a register (bit12=0)
or a imm/offset (bit12=1)

*2: FLAG register is unused (bit11=0)
or used (bit11=1)

ISA of Lab CPU

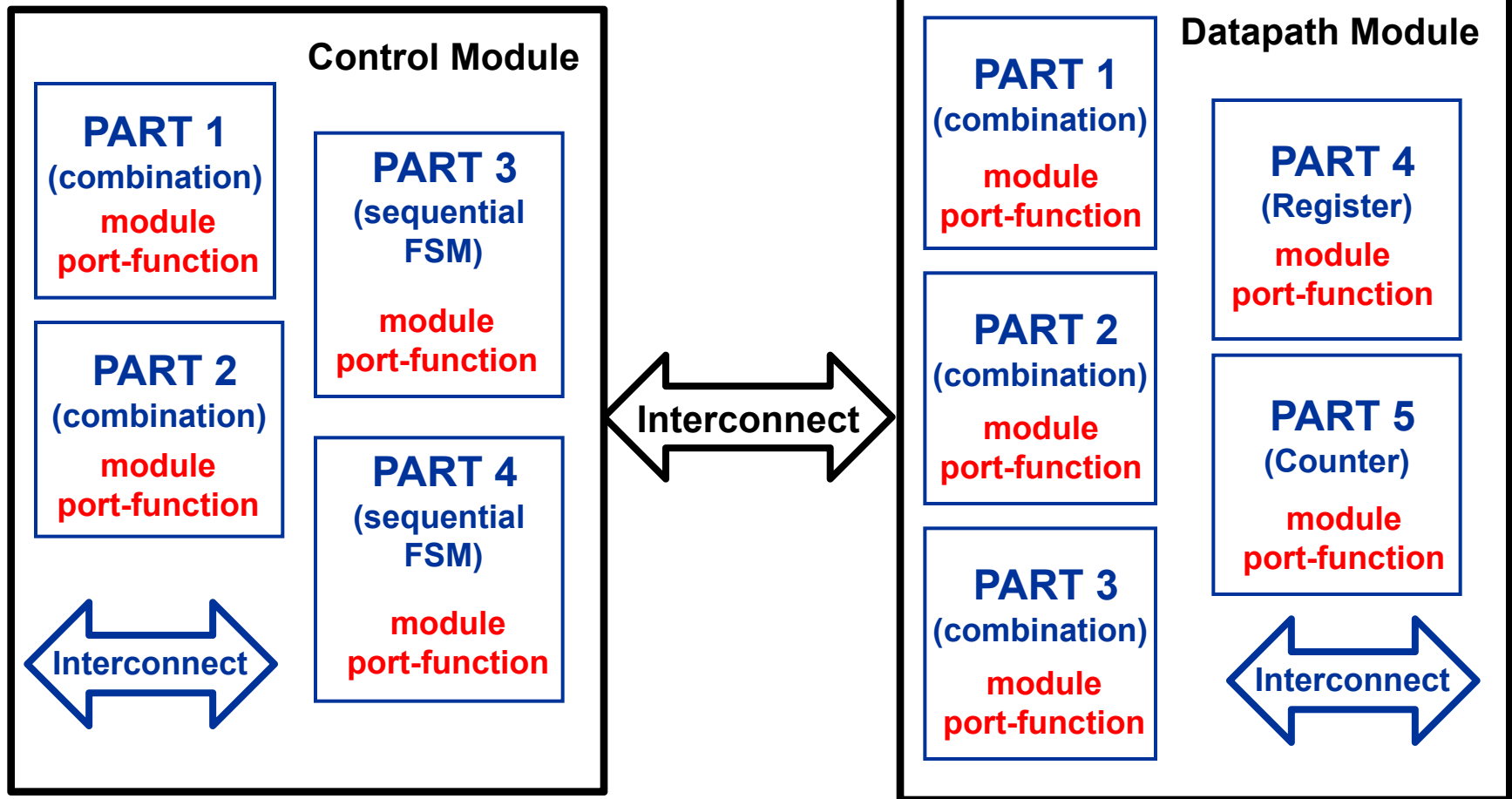
Opcode	Rd	Rs	Imme	assembly	function
4bit	2bit	2bit	8bit		
0000	01	xx	00000010	Mov R1 #02	R1<= 2
0001	10	01	xxxxxxxx	Mov R2 R1	R2<=(R1)
0010	01	xx	00001000	Add R1 #08	R1<=(R1)+8
0011	01	10	xxxxxxxx	Add R1 R2	R1<=(R1)+(R2)
0100	01	10	xxxxxxxx	Sub R1 R2	R1<=(R1)-(R2)
0101	01	10	xxxxxxxx	And R1 R2	R1<= (R1) bit_and (R2)
0110	01	10	xxxxxxxx	Or R1 R2	R1<=(R1) bit_or (R2)
0111	xx	xx	00000001	Jump #01	PC<= 1

- Opcode supports 16 instructions. So you can add other instructions, like shift, multiplication.
- Rd and Rs can be expressed in 2 bits that means you can operate 4 registers.
- Immediate data is expressed in 8 bits.

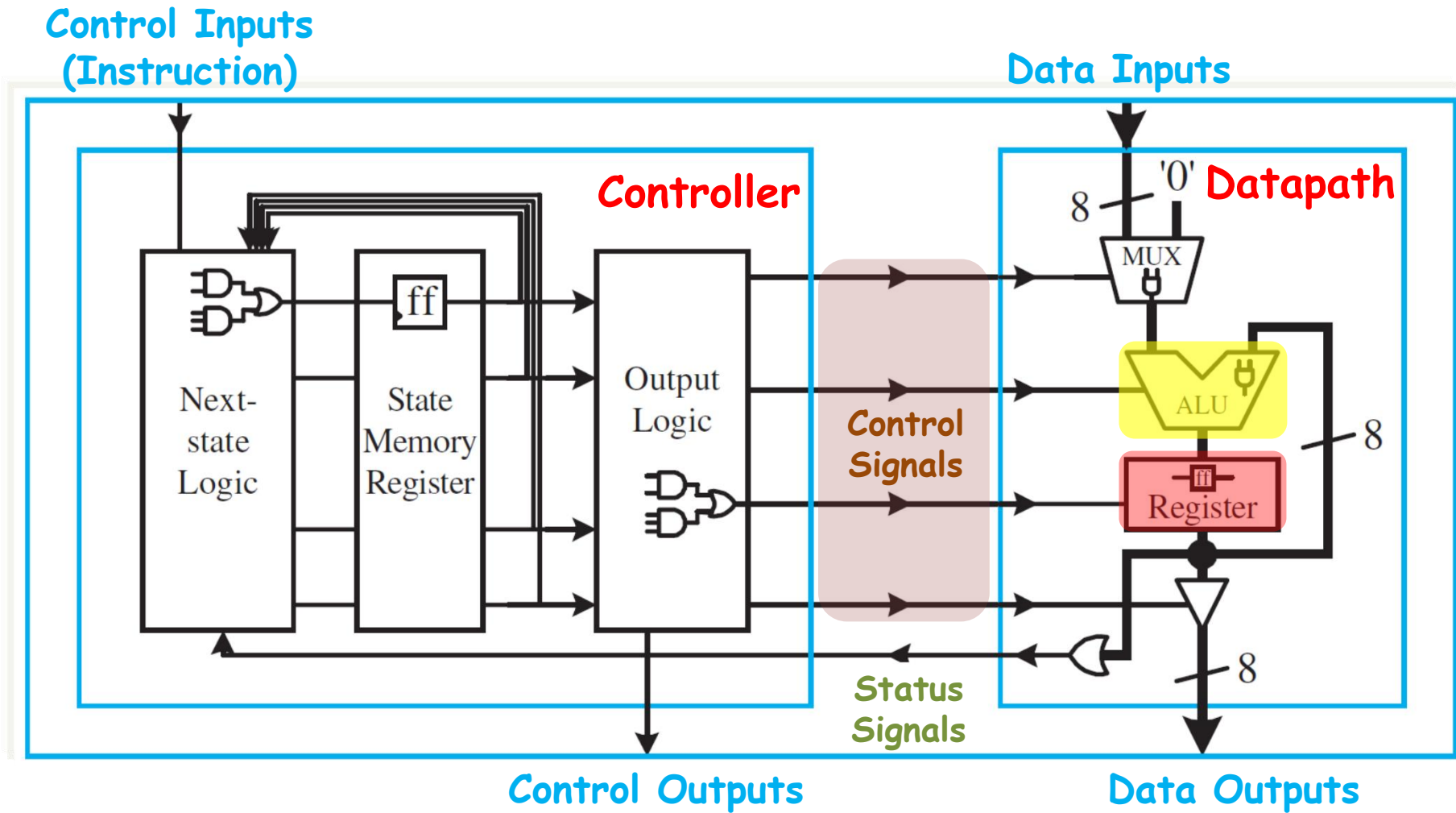
TOP-DOWN Design

Digital circuit

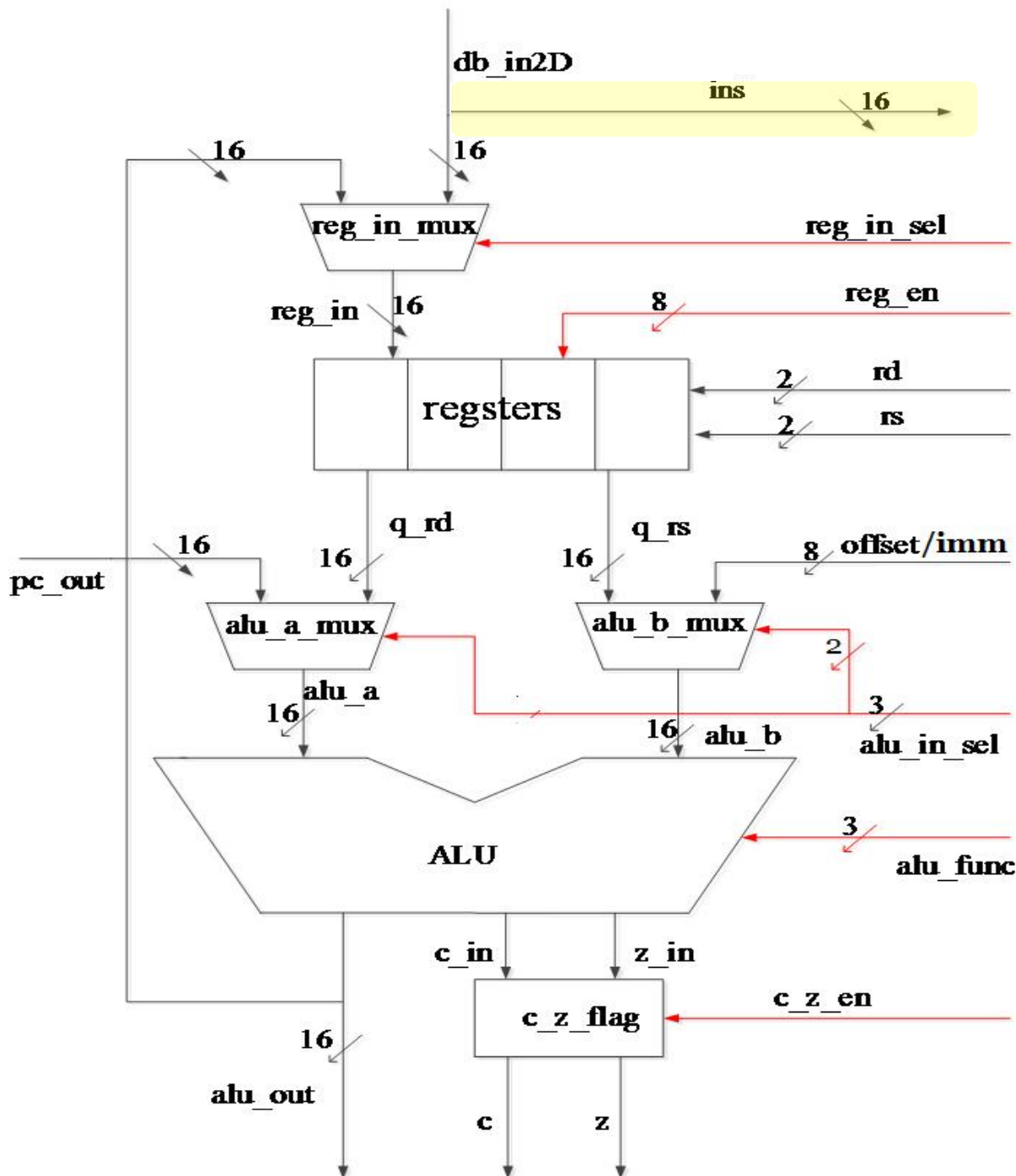
Top-level module port-function



Organization of Center Processor Unit

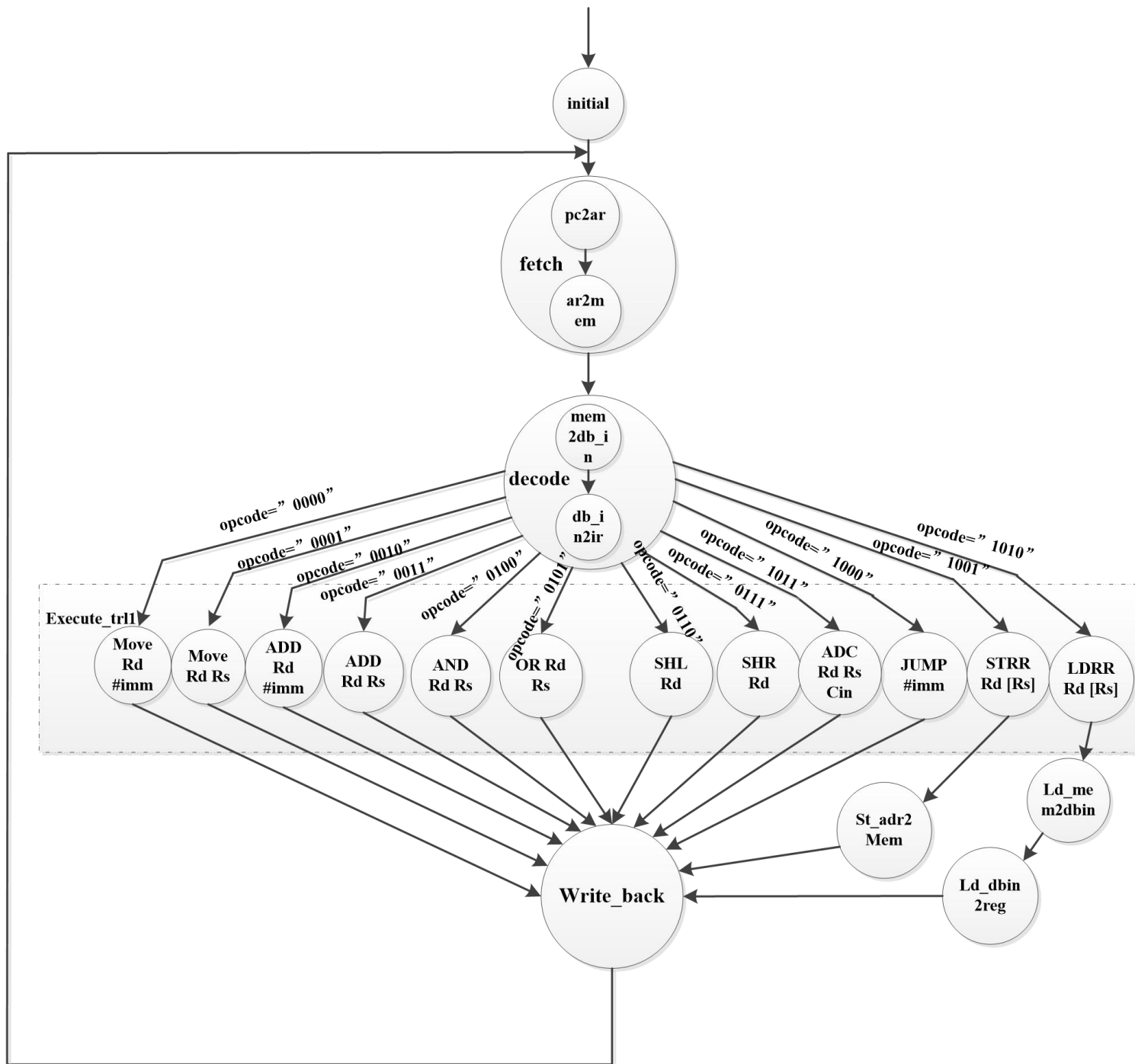


Datapath depends on ISA



MOV R0 , R1
MOV R0 , #0x22
ADD R0 , R1
ADD R0 , #0x22
ADDC R0 , R1
ADDC R0 , #0x22
SUB R0 , R1
SUB R0 , #0x22
SUBB R0 , R1
SUBB R0 , #0x22
AND R0 , R1
NOT R0
JMP R0
JZ XXX
JC XXX
LAD R0, R1
STR R0, R1

STG for Control unit



TestBench Anatomy

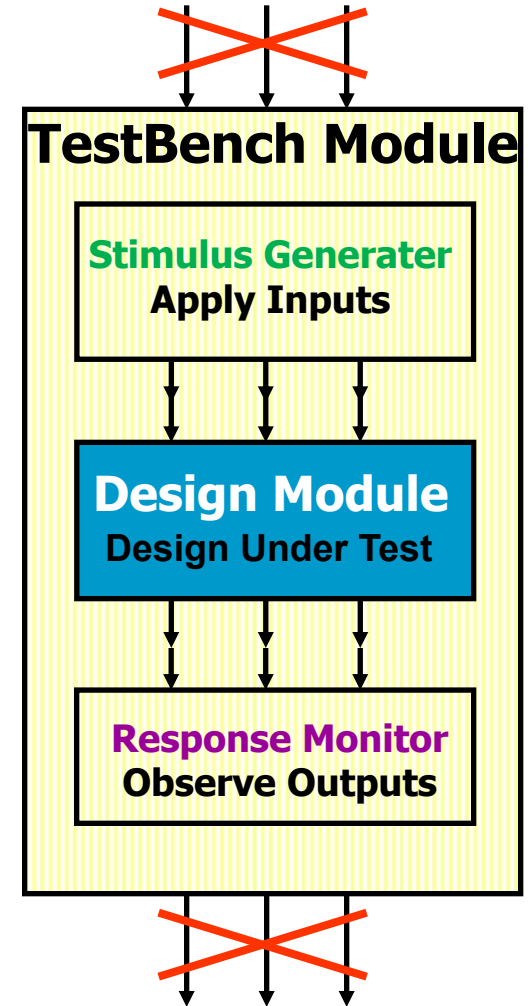
```
module My_tb;                                --TestBench module has no ports
    reg .....;                               --Local signals and constants
    wire .....;

    Instantiating module;  -- Instantiations of DUT

    initial begin ..... end                 -- Input stimuli: reset
    always begin ..... end                  -- Input stimuli: clock
    initial begin ..... end                 -- Input stimuli: other signals

    always @ ( posedge clock )
        begin                               -- Compare results
            .....
        end

    always @ ( posedge clock )
        begin                               -- Display results
            .....
        end
endmodule
```



A Simple Tester

```
`timescale 1ns/100ps
module Detector110Tester;
    reg aa, clock, rst;
    wire ww;
    Detector110 UUT (aa, clock, rst, ww);
    initial begin
        aa = 0; clock = 0; rst = 1;
    end
    initial repeat (44) #7 clock = ~clock;
    initial repeat (15) #23 aa = ~aa;
    initial begin
        #31 rst = 1;
        #23 rst = 0;
    end
    always @(ww) if (ww == 1)
        $display ("A 1 was detected on w at time = %t", $time);
endmodule
```

Time unit

No input or output ports

Inputs & Outputs of DUT

The Instantiation of DUT (Detector110 Module)

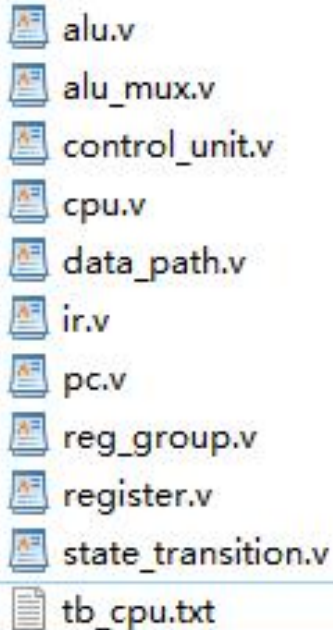
An initial statement

Generates a periodic signal :
repeats (44) times with (7time steps) delay,

Generates a no-periodic
signal

A Verilog System Task: reports the Times
at which the ww Variable becomes 1

Design & Implement for CPU



```

module cpu(clk,rst ,en_in,en_ram_out,addr,ins,en_ram_in );

input      clk,rst ,en_in,en_ram_out;
input  [15:0] ins;
output [15:0] addr;
output      en_ram_in;

wire      en_pc_pulse,en_group_pulse,alu_in_sel,en_alu;
wire  [1:0] pc_ctrl;
wire  [3:0] reg_en;
wire  [2:0] alu_func;
wire  [7:0] offset_addr;

data_path data_path1 (
    .clk(clk),
    .rst(rst),
    .offset(ins[7:0]),
    .offset_addr(offset_addr),
    .en_pc_pulse(en_pc_pulse),
    .pc_ctrl(pc_ctrl),
    .en_in(en_group_pulse) ,
    .reg_en(reg_en) ,
    .rd(ins[11:10]),
    .rs(ins[9:8]),
    .alu_in_sel(alu_in_sel),
    .alu_func(alu_func),
    .en_out(en_alu ),
    .pc_out(addr)
);

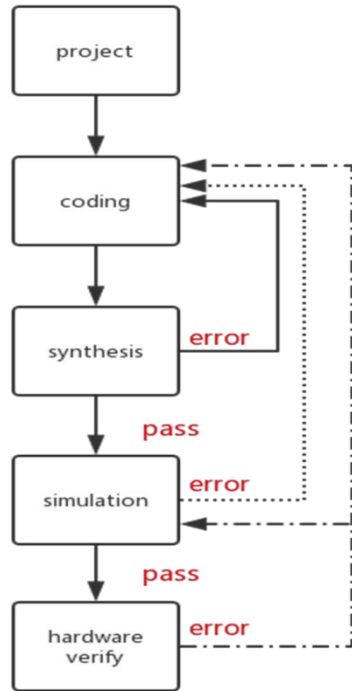
control_unit control_unit1(
    .clk(clk ) ,
    .rst(rst) ,
    .en(en_in) ,
    .en_alu(en_alu ) ,
    .en_ram_out(en_ram_out) ,
    .ins(ins),
    .offset_addr(offset_addr),
    .en_ram_in(en_ram_in ),
    .en_group_pulse(en_group_pulse ),
    .en_pc_pulse(en_pc_pulse),
    .reg_en(reg_en),
    .alu_in_sel(alu_in_sel),
    .alu_func (alu_func),
    .pc_ctrl(pc_ctrl)
);

endmodule
    
```

```

23 module tb_cpu();
24     reg      clk,rst,en_in,en_ram_out;
25     reg  [15:0] ins;
26     wire      en_ram_in;
27     wire  [15:0] addr;
28
29     cpu test_cpu(
30         .clk (clk),
31         .rst (rst),
32         .en_in (en_in),
33         .en_ram_in (en_ram_in),
34         .ins (ins),
35         .en_ram_out (en_ram_out),
36         .addr (addr)
37     );
38
39     parameter Tclk = 10;
40
41     initial begin
42         //define clk
43         end
44
45     initial begin
46         //define rst
47         end
48
49     initial begin
50         //define en_in and en_ram_out
51     end
52
53     initial begin
54         //define ins
55         end
56
57     initial begin
58         #(Tclk*400) $stop;
59     end
60
61 endmodule
    
```


EDA tools & flow



After finished, you can see this window

The screenshot shows the Vivado IDE interface. The left sidebar contains the Project Manager, IP Integrator, and a list of menus. The main window displays the Project Manager for 'project_2'. The 'Sources' tab is active, showing a list of sources: Design Sources, Constraints, Simulation Sources, and sim_1. The 'Simulation' menu is highlighted, showing options like Run Simulation, Open Elaborated Design, and Open Synthesized Design. The 'Synthesis' menu is highlighted, showing options like Run Synthesis, Open Synthesized Design, Run Implementation, and Open Implemented Design. The 'Hardware Program' menu is highlighted, showing options like Generate Bitstream and Open Hardware Manager. The 'Project Summary' window is open, displaying project details: Project name: project_2, Project location: C:\Users\Administrator\project_2, Product family: Zynq-7000, Project part: xc7z020dpg484-1, Top module name: Not defined, Target language: Verilog, Simulator language: Mixed. The 'Synthesis' tab is selected, showing the status: Not started, Messages: No errors or warnings, Part: xc7z020dpg484-1, Strategy: Vivado Synthesis Defaults. The 'Implementation' tab is also visible, showing the status: Not started, Messages: No errors or warnings, Part: xc7z020dpg484-1, Strategy: Vivado Implementation Defaults, Incremental compile: None. The 'Utilization' and 'Power' tabs are also visible. The bottom status bar shows the project name and various icons.

Source Menu

Simulation Menu

Synthesis Menu

Hardware Program Menu

PERFORMANCE ASSESSMENT

Consider the execution of a program which results in the execution of 2 million instructions on a 400-MHz processor. The program consists of four major types of instructions. The instruction mix and the CPI for each instruction type are given below based on the result of a program trace experiment. Please give out the average CPI , the corresponding MIPS and the tuning time T when the program is executed on a uniprocessor .

Instruction Type	CPI	Instruction Mix
Arithmetic and logic	1	60%
Load/store with cache hit	2	18%
Branch	4	12%
Memory reference with cache miss	8	10%

$$\text{CPI} = 0.6 + (2 * 0.18) + (4 * 0.12) + (8 * 0.1) = 2.24$$

$$\text{MIPS} = (400 * 10^6) / (2.24 * 10^6) = 178$$

$$T = 2 * 10^6 * 2.24 / (400 * 10^6) = 0.012 \text{ (s)}$$